ARRAY

1. Which of these best describes an array?
a) A data structure that shows a hierarchical behavior
b) Container of objects of similar types
c) Arrays are immutable once initialized
d) Array is not a data structure

2. How do you initialize an array in C?
a) int arr[3] = (1,2,3);
b) int arr(3) = 1,2,3;
c) int arr[3] = 1,2,3;
d) int arr(3) = (1,2,3);

3. Which of the following concepts make extensive use of arrays?
a) Binary trees
b) Scheduling of processes
c) Caching
d) Spatial locality

4. What are the advantages of arrays?
a) Objects of mixed data types can be stored
b) Elements in an array cannot be sorted
c) Index of first element of an array is 1
d) Easier to store elements of same data type

5. What are the disadvantages of arrays?
a) Data structure like queue or stack cannot be implemented
b) There are chances of wastage of memory space if elements inserted in an array are lesser than the allocated size
c) Index value of an array can be negative
d) Elements are sequentially accessed

6. Assuming int is of 4bytes, what is the size of int arr[15];?
a) 15 b) 19 c) 11 d) 60

7. In general, the index of the first element in an array is _____
a) 0 b) -1 c) 2 d) 1

8. Elements in an array are accessed _____
a) randomly
b) sequentially
c) exponentially
d) logarithmically

9. If an array is declared as arr[] = 1,3,5,7,9; then what is the value of sizeof(arr[3])?
a) 1 b) 2 c) 3 d) 8

10. If an array is declared as arr[] = 1,3,5,7,9; then what is the value of arr[3]?
a) 1 b) 7 c) 9 d) 5

11. If an array is declared as double arr[50]; how many bytes will be allocated to it?
a) 50 b) 100 c) 200 d) 400

12. If an array is declared as int arr[50], how many elements can it hold?
a) 49 b) 50 c) 51 d) 0

13. Given an integer array arr[]; the ith element can be accessed by writing
a) *(arr + i) b) *(i + arr) c) arr[i] d) All of these

14. Given an integer array arr[] having n elements, the following code presents the array insertion in a specic position pos, find the missing code

```
void insert(int n, int arr[],int x, int pos){
      int i;
      n++;
      ......................
      arr[pos - 1] = x;
      return;
}
a) for (i = n-1; i >= pos; i--) arr[i] = arr[i - 1];
b) for (i = n; i >= pos; i--) arr[i] = arr[i - 1];
c) for (i = n-1; i > pos; i--) arr[i] = arr[i - 1];
d) for (i = n; i > pos; i--) arr[i] = arr[i - 1];
```

15. Find missing condition of the following function implementing for search operation.

```
int findElement(int arr[], int n,int key){
      int i;
      for (i = 0; i < n; i++)
      if (.................)
            return i;
      return -1;
}
a)arr[i] != key   b)arr[i] > key    c)arr[i] == key   d)arr[i] >= key
```

16. Find missing statement of the following function implementing for deletion operation of specific position pos.

```
int deleteElement(int arr[], int n, int key, int pos){
      int i;
      ........................
```

```
      return n - 1;
}
```
a) `for(i=pos-1; i<=n-1; i++) arr[i] = arr[i + 1];`
b) `for(i=pos; i<=n-1; i++) arr[i] = arr[i + 1];`
c) `for(i=pos-1; i<n-1; i++) arr[i] = arr[i + 1];`
d) `for(i=pos; i<n-1; i++) arr[i] = arr[i + 1];`

17. Given the base address of an array A[1300 ...1900] as 1020 and the size of each element is 2 bytes in the memory, the address of A[1700] is
a) 2000 b) 1800 c) 1820 d) 1840

18. Given an array, arr[1...10][1...15] with base value 100 and the size of each element is 1 Byte in memory. The address of arr[8][6] with the help of row-major order is.....
a) 200 b) 210 c) 220 d) 180

19. Given an array, arr[1...10][1...15] with base value 100 and the size of each element is 1 Byte in memory. The address of arr[8][6] with the help of column-major order is.....
a) 200 b) 210 c) 150 d) 157

20. Predict output of following program
```
int main(){
      int i;
      int arr[5] = {1};
      for (i = 0; i < 5; i++)
            printf("%d ", arr[i]);
      return 0;
}
```
a) 1 followed by four garbage values:
b) 1 0 0 0 0
c) 1 1 1 1 1
d) 0 0 0 0 0

21. Predict output of following program
```
int main(){
      int a[][] = {{1,2},{3,4}};
      int i, j;
      for (i = 0; i < 2; i++)
            for (j = 0; j < 2; j++)
                  printf("%d ", a[i][j]);
            return 0;
}
```
a) 1 2 3 4
b) Compiler Error in line int a[][] = 1,2,3,4;
c) 4 garbage values
d) 4 3 2 1

22. What does the following function do for a given Linked List with the first node as head?

```
void fun1(struct node* head){
      if(head == NULL)
            return;
      fun1(head->next);
      printf("%d ", head->data);
}
```

a) Prints all nodes of linked lists

b) Prints all nodes of the linked list in reverse order

c) Prints alternate nodes of Linked List

d) Prints alternate nodes in reverse order

23. What is the output of the following function if start points to the first node of the linked list
1 → 2 → 3 → 4 → 5 → 6?

```
void fun(struct Node* start){
      if(start == NULL) return;
      printf("%d ", start->data);
      if(start->next != NULL ) fun(start->next->next);
      printf("%d ", start->data);
}
```

a) 1 4 6 6 4 1

b) 1 3 5 1 3 5

c) 1 2 3 5

d) 1 3 5 5 3 1

1. What is the importance of data structure in computer science?

- **Efficiency:** Data structures allow for the efficient storage and organization of data, which directly impacts the performance (speed and memory usage) of a software application.
- **Abstraction:** They provide a level of abstraction that allows programmers to manage complexity. Large datasets can be handled as single entities (like a generic "List" or "Tree") rather than individual memory addresses.
- **Reusability:** Standard data structures are reusable components. Once implemented, a stack or a linked list can be used in various parts of a program or in different programs entirely.
- **Algorithmic Foundation:** Most efficient algorithms (like sorting, searching, or graph traversal) rely heavily on specific data structures to function correctly. For example, BFS uses a Queue, while DFS uses a Stack.

2. Differentiate data structure, algorithm and program with a suitable example.

- **Data Structure:** This is the *organization* of data. It describes how data is stored and related in memory.
    - *Example:* An array A[10] that stores 10 integers sequentially.
- **Algorithm:** This is the *recipe* or step-by-step procedure to solve a problem. It is platform-independent logic.

    - *Example:* Binary Search logic (Divide and Conquer) to find a number in a sorted list.
- **Program:** This is the *implementation*. It is the concrete code (in C, Python, etc.) that combines data structures and algorithms to run on a computer.
    - *Example:* A C code file search.c that declares an array and implements the binary search loop to find a user-inputted value.

3. What are the types of data structure? What are the need and applications of data structure?

Types of Data Structures:

- **Primitive DS:** Basic structures supported directly by the machine (e.g., Integer, Float, Char, Pointer).
- **Non-Primitive DS:** Derived from primitive structures.
    - **Linear:** Elements are arranged sequentially (e.g., Arrays, Linked Lists, Stacks, Queues).
    - **Non-Linear:** Elements are arranged hierarchically or connected (e.g., Trees, Graphs).

Need and Applications:

- **Data Management:** Essential for Operating Systems (File Systems use Trees) and Database Management Systems (B-Trees for indexing).
- **Resource Sharing:** Printer spoolers uses Queues to manage print jobs.
- **Network Routing:** Graphs are used to find the shortest path in GPS and network routing protocols.
- **Undo Mechanisms:** Text editors use Stacks to implement Undo/Redo operations.

4. What is Abstract Data type? Explain Abstract Data type (ADT) with examples.

- **Definition:** An Abstract Data Type (ADT) is a logical description of a data structure. It defines *what* the data structure does (the operations and the data) but not *how* it is implemented in code. It hides the complexity of the implementation from the user.
- **Components:** An ADT consists of:
    1. **Data:** The information to be stored.
    2. **Operations:** The functions that can be performed on the data.
- **Example (Stack ADT):**
    - *Data:* A collection of elements.
    - *Operations:* push(x) (add x to top), pop() (remove top), peek() (view top), isEmpty() (check if empty).
    - *Independence:* The user of the Stack ADT doesn't need to know if the underlying code uses an Array or a Linked List; they only use the push and pop operations.

5. Can a primitive data type be considered as an ADT?

- **Yes, in a conceptual sense.** Primitive data types like int or float can be viewed as ADTs because they have a defined set of values (Data) and a defined set of

operations (Addition, Subtraction, Multiplication) that can be performed on them.

- **Abstraction:** When a programmer uses an int, they generally do not worry about the underlying binary representation (2's complement) or the specific CPU registers used for addition. They simply use the defined operations. However, in formal DS contexts, ADT usually refers to user-defined complex structures.

6. List out some of the areas in which data structures are used?

- **Compiler Design:** Uses Hash Tables for symbol tables and Stacks for syntax parsing.
- **Operating Systems:** Uses Queues for process scheduling and Lists for memory management.
- **Database Management Systems:** Uses B-Trees and B+ Trees for efficient data indexing and retrieval.
- **Artificial Intelligence:** Uses Graphs and Trees for state-space search and decision making.
- **Graphics:** Uses Matrices for transformations and Trees (Quadtrees/Octrees) for rendering scenes.

7. What are the major data structures used in the following areas: RDBMS, Network data model and Hierarchical data model?

- **RDBMS (Relational Database Management System):**
  - **B-Trees and B+ Trees:** Primarily used for indexing data to ensure efficient retrieval (searching, insertion, and deletion) which typically takes logarithmic time.
  - **Hash Tables:** Used for implementing associative arrays and database indexes where equality searches are common.
- **Network Data Model:**
  - **Graphs:** The network model views data as a set of records connected by links (relationships). A Graph data structure is the natural representation, allowing many-to-many relationships (nodes connected by edges).
- **Hierarchical Data Model:**
  - **Trees:** This model organizes data in a tree-like structure with a single root, where records have a 1:N (parent-child) relationship.

8. Define an algorithm. What are the properties of an algorithm?

- **Definition:** An algorithm is a finite, step-by-step sequence of well-defined instructions to solve a specific problem or perform a computation.
- **Properties:**
  - **Input:** It must accept zero or more inputs.
  - **Output:** It must produce at least one output.
  - **Definiteness:** Each instruction must be clear and unambiguous.
  - **Finiteness:** The algorithm must terminate after a finite number of steps.

- **Effectiveness:** The steps must be basic enough to be carried out effectively.

9. Which properties of an algorithm distinguish it from a program?

- **Independence:** An algorithm is a design or logic concept that is independent of computer hardware and programming languages. A program is the specific implementation of that logic for a specific machine.
- **Finiteness:** An algorithm must strictly terminate. A program (like an Operating System) may theoretically run indefinitely (infinite loop) waiting for events.
- **Analysis vs. Syntax:** Algorithms are analysed for time/space complexity (O-notation), whereas programs are checked for syntax and semantic correctness by a compiler.

10. What is the Brute Force algorithm?

- **Definition:** A Brute Force algorithm is a straightforward approach to solving a problem that relies on sheer computing power rather than advanced techniques to improve efficiency.
- **Method:** It typically involves iterating through all possible solutions to check which one satisfies the problem constraints.
- **Examples:** Linear Search (checking every element), generating all permutations of a string.
- **Characteristics:** Usually simple to implement but often inefficient (high time complexity) for large datasets.

11. What values are automatically assigned to array elements which are not explicitly initialized?

- **Local/Automatic Arrays:** If defined inside a function without the static keyword, uninitialized elements contain **garbage values** (random contents of the memory location at that time).
- **Static/Global Arrays:** If defined outside all functions or with the static keyword, uninitialized elements are automatically initialized to **Zero (0)** (or NULL for pointers).

12. What are the limitations of arrays? How can you overcome them? Why are they important despite these limitations?

- **Limitations:**
  - **Fixed Size:** Size must be known at compile time (in C89/static allocation) and cannot be resized dynamically.
  - **Insertion/Deletion:** Expensive operations O(n) because elements must be shifted to maintain contiguous storage.
  - **Memory Wastage:** Allocating a large array for a few elements wastes memory (internal fragmentation).
- **How to Overcome:**
  - Use **Dynamic Arrays** (e.g., using malloc/realloc in C) to resize at runtime.

    - Use **Linked Lists** which allow dynamic memory allocation and efficient insertion/deletion O(1) without shifting.
- **Importance:**
    - **Random Access:** Arrays provide O(1) access time to any element using the index, which is critical for performance.
    - **Cache Friendliness:** Contiguous memory allows for better spatial locality, improving CPU cache performance compared to Linked Lists.

13. Explain an efficient way of storing two symmetric matrices of the same order in memory.

- **Concept:** A symmetric matrix A has A[i][j] = A[j][i]. Storing the full matrix involves redundant data. We only need to store the lower triangular (or upper triangular) part.
- **Storage Method:**
    - Use a **1-D Array** of size N(N+1)/2 to store the Lower Triangular part of the first matrix.
    - Use a second **1-D Array** of the same size for the second matrix.
- **Mapping Formula (Row-Major Lower Triangular):**
    - The address for element A[i][j] (where i ≥ j) is calculated as: Address(A[i][j]) = $Base + Size * [\frac{i(i-1)}{2} + j]$.
- **Alternative (Packing):** Sometimes, two symmetric matrices can be packed into a single square matrix C, where Matrix 1 occupies the lower triangle and Matrix 2 occupies the upper triangle (if the diagonal is handled separately or shared).

14. How can data be organized in memory? What are static and dynamic memory allocations? Explain with an example.

- **Data Organization:** Data is organized either **sequentially** (contiguous addresses, e.g., Arrays) or **non-sequentially** (linked via pointers, e.g., Linked Lists).
- **Static Memory Allocation:**
    - Memory is allocated at **compile time**.
    - Size is fixed.
    - Stored in the **Stack**.
    - *Example:* int arr[10]; (Allocates 40 bytes instantly).
- **Dynamic Memory Allocation:**
    - Memory is allocated at **run time**.
    - Size can be determined during execution.
    - Stored in the **Heap**.
    - *Example:* `int *ptr = (int*)malloc(n*sizeof(int));` (Allocates memory based on user input n).

15. What is a sparse matrix? How is it represented in memory? What are the types of sparse matrices?

- **Definition:** A matrix is called sparse if the majority of its elements are zero.
- **Representation:** Storing standard 2D arrays wastes space.

    - **Triplet Representation (Array):** A 2D array where each row is {Row, Col, Value}. Only non-zero elements are stored.
    - **Linked List Representation:** Each node contains the data and pointers to the next node in the row and column.
- **Types:**
    - **Triangular Matrix:** Non-zero entries only above/below the diagonal.
    - **Band Matrix:** Non-zero entries confined to a diagonal band.
    - **Diagonal Matrix:** Non-zero entries only on the main diagonal.

16. What are the different characteristics of an algorithm? Briefly explain.

- **Input:** The algorithm must have well-defined inputs (0 or more).
- **Output:** It must produce at least one defined output.
- **Definiteness:** Every step must be clear, unambiguous, and precise.
- **Finiteness:** It must terminate after a finite number of steps; it cannot run forever.
- **Effectiveness:** Operations must be basic enough to be performed exactly and in a finite length of time by a person or machine.

17. What is a subscripted variable? Differentiate between Primitive Data Structure and Primitive Data-Type.

- **Subscripted Variable:** A variable that refers to a specific element of an array. It consists of the array name followed by an index (subscript) in brackets (e.g., Arr[5]).
- **Primitive Data Structure:** The theoretical/conceptual building blocks of data organization (e.g., Integer, Real, Boolean, Character). They are the atomic units of data.
- **Primitive Data Type:** The practical implementation of these structures in a specific programming language (e.g., int, float, char in C). It defines the specific size (bytes) and range of values the hardware can handle.

18. Write an algorithm to find the smallest element in the array.

- **Step 1:** Start.
- **Step 2:** Declare an array Arr of size N.
- **Step 3:** Initialize a variable min with the value of the first element (min = Arr[0]).
- **Step 4:** Set a loop counter i = 1.
- **Step 5:** Repeat steps 6 and 7 while i < N.
- **Step 6:** If Arr[i] < min, then update min = Arr[i].
- **Step 7:** Increment i by 1.
- **Step 8:** Print "Smallest element is " min.
- **Step 9:** Stop.

19. What is the addressing formula for an element A[i][j] in row major order, if i and j are bounded by the limits $l_1 \leq i \leq u_2$ and $l_2 \leq j \leq u_2$? Assume that the base address is L and w is the number of words allocated to each element.

- **Parameters:**
  - Base Address: L
  - Storage size per element: w
  - Number of columns (N): $\boldsymbol{u_2 - l_2 + 1}$
- **Formula:**

$$\boldsymbol{Address(A[i][j]) = L + w \times [(i - l_1) \times N + (j - l_2)]}$$

- **Explanation:**
  - $(i - l_1)$: Calculates the number of full rows to skip before the current row.
  - Multiply by N: Converts those rows into the total number of elements.
  - $(j - l_2)$: Adds the offset of the column in the current row.

20. Write an algorithm to multiply two two-dimensional arrays of size m x n. (Note: Matrix multiplication is only possible if the columns of the first matrix equal the rows of the second. Assuming valid dimensions A[m][p] and B[p][n] to produce C[m][n])

- **Step 1:** Initialize a result matrix C[m][n] with all elements set to 0.
- **Step 2:** Start a loop for i from 0 to m-1 (iterate rows of A).
- **Step 3:** Inside, start a loop for j from 0 to n-1 (iterate columns of B).
- **Step 4:** Inside, start a loop for k from 0 to p-1 (iterate common dimension).
- **Step 5:** Calculate C[i][j] = C[i][j] + (A[i][k] * B[k][j]).
- **Step 6:** End inner loops.
- **Step 7:** Print Matrix C.

21. Given an array A[10], initially containing six elements, write a C function to insert a new element at the third position.

```
void insertAtThird(int A[], int *n, int value) {
    // *n is current number of elements (6), max size is 10
    if (*n >= 10) {
        printf("Array is full.\n");
        return;
    }
    // Shift elements from index 2 (3rd pos) to end one step right
    // 3rd position corresponds to index 2
    for (int i = *n; i > 2; i--) {
        A[i] = A[i-1];
    }
    A[2] = value; // Insert at index 2
    (*n)++;       // Increase count
}
```

22. Given an array A[10], initially containing six elements, write a C function to delete an element at the third position.

```
void deleteAtThird(int A[], int *n) {
    // *n is current number of elements (6)
    if (*n < 3) {
```

```
        printf("Position does not exist.\n");
        return;
    }
    // Shift elements from index 3 to end one step left to overwrite index
2
    // 3rd position corresponds to index 2
    for (int i = 2; i < *n - 1; i++) {
        A[i] = A[i+1];
    }
    (*n)--; // Decrease count
}
```

23. Given an array B[10], initially containing 8 elements, write a C function to delete an element. If the element does not exist, display an appropriate message.

```
void deleteElement(int B[], int *n, int key) {
    int i, pos = -1;
    // Search for the element
    for (i = 0; i < *n; i++) {
        if (B[i] == key) {
            pos = i;
            break;
        }
    }
    if (pos != -1) {
        // Shift elements to the left
        for (i = pos; i < *n - 1; i++) {
            B[i] = B[i+1];
        }
        (*n)--;
        printf("Element deleted successfully.\n");
    } else {
        printf("Element not found.\n");
    }
}
```

24. Let A be a two-dimensional array declared as follows: A : array[1...10][1...15] of integer. Assuming each integer occupies one memory location and the array is stored in row-major order with the first element stored at location 100, find the address of element A[i][j].

- **Given:**
  - Base Address (L) = 100
  - Lower Bound Row (l_r) = 1
  - Lower Bound Col (l_c) = 1
  - Number of Cols (N) = 15
  - Size (w) = 1
- **Formula:**

$$\boldsymbol{Address(A[i][j]) = L + w \times [(i - l_1) \times N + (j - l_2)]}$$

- **Calculation:**

$$\boldsymbol{Address(A[i][j]) = 100 + 1 \times [(i - 1) \times 15 + (j - 1)]}$$
$$\boldsymbol{Address(A[i][j]) = 100 + 15(i - 1) + (j - 1)}$$

25. Given an 8 × 3 matrix with elements of size 4 bytes each and a base address of 3500, find the address of A[5, 2] in both row-major and column-major order.

- **Given:** Rows (M)=8, Cols (N)=3, Size (w)=4, Base (L)=3500. Assuming 0-based indexing (l_r=0l_c, =0).
- **Row-Major Order:**
  - Formula: $\boldsymbol{L + w \times (i \times N + j)}$
  - $\boldsymbol{3500 + 4 \times (5 \times 3 + 2)}$
  - $\boldsymbol{3500 + 4 \times (17) = 3500 + 68}$
  - **Result: 3568**
- **Column-Major Order:**
  - Formula: $\boldsymbol{L + w \times (j \times M + i)}$
  - $\boldsymbol{3500 + 4 \times (2 \times 8 + 5)}$
  - $\boldsymbol{3500 + 4 \times (21) = 3500 + 84}$
  - **Result: 3584**

26. Suppose a 2-dimensional array defined as A[4...7, −1...3] requires 2 bytes of storage per element. If the array is stored in row-major form, calculate the address of A[6, 2] with a base address of 100. Repeat for column-major form.

- **Given:**
  - Base (L) = 100, Size (w) = 2.
  - Rows (l_r=4, u_r=7). Total Rows (M) = 7-4+1 = 4.
  - Cols (l_c= -1, u_c=3). Total Cols (N) = 3-(-1)+1 = 5.
  - Target: i=6, j=2.
- **Row-Major Order:**
  - Formula: $\boldsymbol{L + w \times [(i - l_r) \times N + (j - l_c)]}$
  - $\boldsymbol{100 + 2 \times [(6 - 4) \times 5 + (2 - (-1))]}$
  - $\boldsymbol{100 + 2 \times [2 \times 5 + 3] = 100 + 2 \times [13]}$
  - **Result: 126**
- **Column-Major Order:**
  - Formula: $\boldsymbol{L + w \times [(j - l_c) \times M + (i - l_r)]}$
  - $\boldsymbol{100 + 2 \times [(2 - (-1)) \times 4 + (6 - 4)]}$
  - $\boldsymbol{100 + 2 \times [3 \times 4 + 2] = 100 + 2 \times [14]}$
  - **Result: 128**

27. For a lower triangular matrix in compact single-dimensional array representation, find the index of the (i, j)th element in this representation.

- **Definition:** In a lower triangular matrix, non-zero elements exist only where row $r \geq col\ c$.
- **Row-Major Mapping:** We store row 0 (1 element), then row 1 (2 elements), then row 2 (3 elements), etc.
- **Formula:** To find the index k for element (i, j) (assuming 0-based indexing): Sum of elements in previous rows (0 to i-1) + column index j.

$$\boldsymbol{Index(k) = \frac{i(i+1)}{2} + j}$$

Note: If using 1-based indexing, the formula is

$$\boldsymbol{\frac{i(i+1)}{2} + (j - 1)}$$

28. A program reads in 500 integers in the range [0...100] representing scores of 500 students and prints the frequency of each score above 50. Propose an efficient storage method for the frequencies.

- **Proposed Method:** Use a **Frequency Array (Hash Map approach)**.
- **Implementation:**
  - Since the range of scores is small (0 to 100), create an integer array freq[101] initialized to 0.
  - Iterate through the 500 scores. For every score s, increment freq[s].
  - To print, iterate from index 51 to 100.
- **Efficiency:** This allows O(1) access to update counts and O(N) to process the input, which is faster than sorting or nested loops.

29. Carly wants to store details of 1st-year students and retrieve the highest marks in each subject. Suggest whether to use a single 2-D array or multiple 1-D arrays.

- **Recommendation:** Use a **Structure (struct) array** or **Multiple 1-D arrays** (Parallel Arrays), NOT a single 2-D array.
- **Reasoning:**
  - A single 2-D array is homogeneous; it can only store one data type (e.g., all integers). Student details usually mix types (Name char[], Roll No int, Marks float).
  - To retrieve highest marks efficiently, parallel 1-D arrays (e.g., Marks[N]) allow direct iteration.
  - **Best Approach in C:** Define a struct Student { char name[50]; int roll; float marks; }; and create an array of these structures.

2. Dave is developing Campus Management Software but faces memory wastage due to over-provisioning in arrays. Propose a suitable memory storage method and implementation in C/C++.

- **Proposed Method:** Use **Dynamic Memory Allocation** (Linked Lists or Dynamic Arrays).
- **Implementation (Dynamic Array):**
  - Instead of int students[1000];, use a pointer int *students;.
  - Ask for the exact number of students (N) at runtime.
  - Allocate memory: students = (int*) malloc(N * sizeof(int));.
- **Implementation (Linked List):**
  - If the number of students changes frequently (admissions/withdrawals), use a Linked List. This allocates memory exactly when a new node is added, eliminating internal fragmentation and pre-allocation waste.

3. Given an array, A, with n unique integers from range x to y (where n = y - x), design a faster algorithm to find the missing number in A.
(Note: The count of integers from x to y inclusive is y-x+1. If the array size is n = y-x, exactly one number is missing.)

- **Algorithm (Summation Method):**
  - **Step 1:** Calculate the expected sum of all integers in the range [x, y].

    $$\boldsymbol{ExpectedSum} = (\boldsymbol{Count}) \times \frac{\boldsymbol{x} + \boldsymbol{y}}{\mathbf{2}}$$

    (Where Count is y - x + 1).
  - **Step 2:** Calculate the ActualSum by iterating through the array A and adding all elements.
  - **Step 3:** The missing number is ExpectedSum - ActualSum.
- **Complexity:**
  - **Time:** $O(n)$ (Linear scan to sum the array).
  - **Space:** $O(1)$ (Only variables for sums are stored).
  - **Why it's faster:** It avoids sorting ($O(n \log n)$) or nested loops ($O(n^2)$).

Asymptotic Notation

1. Which of the following best describes the meaning of $O(f(n))$?
(a) The set of all functions that are bounded below by f(n)
(b) The set of all functions that are asymptotically less than or equal to f(n)
(c) The set of all functions that are asymptotically greater than or equal to f(n)
(d) The set of all functions that are asymptotically equal to f(n)

2. If $T(n) = 2n^2 + 3n + 1$, which of the following is correct?
(a) $T(n) = O(n)$
(b) $T(n) = O(n^2)$
(c) $T(n) = O(n^3)$
(d) $T(n) = \Theta(n)$

3. What does $\Theta(f(n))$ represent?
(a) The set of functions that grow faster than f(n)
(b) The set of functions that grow slower than f(n)
(c) The set of functions that grow at the same rate as f(n)
(d) The set of functions that are unrelated to f(n)

4. Which of the following is true if $f(n) = O(g(n))$?
(a) f(n) grows faster than g(n)
(b) f(n) grows slower than g(n)
(c) f(n) grows at the same rate as g(n)
(d) f(n) is asymptotically equal to g(n)

5. If $f(n) = 3n^3 + 2n^2 + n + 1$, what is the asymptotic notation for f(n)?
(a) $O(n^2)$
(b) $O(n^3)$
(c) $O(n^4)$
(d) $O(n)$

6. Which notation describes the upper bound of an algorithm's running time?
(a) O-notation
(b) Ω-notation
(c) Θ-notation
(d) o-notation

7. For $T(n) = 5n \log n + 20n$, which of the following is true?
(a) $T(n) = O(n)$
(b) $T(n) = O(n \log n)$
(c) $T(n) = O(n^2)$
(d) $T(n) = \Theta(n)$

8. Which of the following best describes Ω(f(n))?
(a) The set of functions that are asymptotically less than or equal to f(n)
(b) The set of functions that are asymptotically greater than or equal to f(n)
(c) The set of functions that are asymptotically equal to f(n)
(d) The set of functions that are asymptotically unrelated to f(n)

9. What does o(f(n)) represent?
(a) Functions that grow slower than f(n)
(b) Functions that grow faster than f(n)
(c) Functions that grow at the same rate as f(n)
(d) Functions that do not grow

10. If T (n) = 7n + 3 and f(n) = n, which of the following is true?
(a) T (n) = $O(1)$
(b) T (n) = $O(n)$
(c) T (n) = $O(n^2)$
(d) T (n) = $\Omega(n^2)$

11. If f(x) = $(x^3 - 1)/(3x + 1)$ then f(x) is?
a)$O(x^2)$ b)$O(x)$ c)$O(x^{1.5}/3)$ d)$O(1)$

12. If f(x) = $3x^2 + x^2 \log x$, then f(x) is?
a)$O(x^2)$ b)$O(x^3)$ c)$O(x)$ d)$O(1)$

13. The big-O notation for f(n) = $(n\log n + n^2)(n^3 + 2)$ is?
a)$O(n^2)$ b)$O(n^3)$ c)$O(n^4)$ d)$O(n^5)$

14. The Θ notation for function f(n) = $2n^3 + n - 1$ is?
a)n b)n^2 c)n^3 d)n^4

15. The Θ notation for f(n) = $n\log(n^2 + 1) + n^2 \log n$ is?
a)$n^2 \log n$ b)n^2 c)$\log n$ d)$n\log(n^2)$

16. The ω notation for f(x, y) = $x^5 y^3 + x^4 y^4 + x^3 y^5$ is?
a) $x^5 y^3$ b) $x^5 y^5$ c) $x^3 y^3$ d) $x^4 y^4$

17. The Big – O notation for f(x) = xlogx is?
a) x b) 1 c)x^2 d) xlogx

18. The big-O notation for f(n) = $2\log(n!) + (n^2 + 1)\log n$ is?
a)n b)n^2 c)nlogn d)$n^2 \log n$

19. The Ω notation for f(x) = 5logx is?
a) 1 b) x c) x^2 d)x^3

20. The O notation for $f(x) = 2x^4 + x^2 - 4$ is?

a)x b)x^2 c)x^3 d)x^4

Stack and Queue

1. What is stack? Why it is called LIFO? What is top of the stack?

- **Stack:** A Stack is a linear data structure that follows a specific order for inserting and deleting elements. It allows operations to be performed at only one end.
- **LIFO (Last In First Out):** It is called LIFO because the element that is inserted last is the first one to be removed. Imagine a stack of plates; you can only take the top plate off, which was the last one you put there.
- **Top of the Stack:** The "Top" is a pointer or index indicating the current position of the most recently added element. All insertion (Push) and deletion (Pop) operations happen strictly at this Top position.

2. What are the conditions for stack underflow & stack overflow?

These conditions normally apply to the array implementation of a stack in C:

- **Stack Overflow:** Occurs when we try to push an element into a stack that is already full.
    - **Condition:** If top == MAX_SIZE - 1 (where MAX_SIZE is the defined capacity of the array).
- **Stack Underflow:** Occurs when we try to pop an element from an empty stack.
    - **Condition:** If top == -1 (assuming the stack is initialized with top at -1).

3. Why stack is called ADT?

- **Definition:** ADT stands for **Abstract Data Type**.
- **Reasoning:** A Stack is called an ADT because it is defined by its logical behavior and operations (Push, Pop, Peek, isEmpty) rather than its implementation details.
- **Independence:** The user knows *what* the stack does, but the underlying mechanism (whether it is implemented using a static Array or a dynamic Linked List) is hidden (abstracted) from the user.

4. What are the applications of stacks?

- **Expression Conversion:** Converting Infix expressions to Postfix or Prefix notations.
- **Expression Evaluation:** Evaluating Postfix or Prefix expressions.
- **Recursion:** The system uses a "Call Stack" to handle function calls and return addresses during recursion.
- **Parsing:** Syntax parsing in compilers (checking for balanced parentheses or braces).
- **Undo Mechanisms:** Used in text editors to track changes (Ctrl+Z).
- **Backtracking Algorithms:** Used in maze solving or DFS (Depth First Search) in graphs.

5. Differentiate between an array and a stack.

Feature	Array	Stack
Access	Allows random access (elements can be accessed via index).	No random access. Only the top element is accessible.
Insertion/Deletion	Possible at any index (though shifting may be required).	Strictly limited to one end (the Top).
Principle	Collection of elements stored in contiguous memory.	Follows LIFO (Last In First Out) principle.
Visibility	All elements are visible.	Only the top element is visible/accessible at a time.

6. Parenthesis are never needed in prefix or postfix expressions. Why?

- **Position Determines Order:** In Prefix (Polish) and Postfix (Reverse Polish) notations, the position of the operator relative to the operands inherently defines the order of evaluation.
- **No Ambiguity:** Unlike Infix notation, where operator precedence (e.g., multiplication before addition) and associativity rules require parentheses to override order, Prefix and Postfix are unambiguous. The operator immediately acts on the adjacent operands required by its arity.

7. Write an algorithm to reverse a queue using a stack.

Algorithm:

1. Initialize an empty Stack S and a Queue Q containing elements.
2. **While** Q is not empty:
    - Dequeue an element x from Q.
    - Push element x onto Stack S.
3. **While** S is not empty:
    - Pop an element x from S.
    - Enqueue element x back into Q.
4. **End:** Queue Q is now reversed.

8. What is queue? Why queue is called FIFO?

- **Queue:** A Queue is a linear data structure that is open at both ends. Data is inserted at one end (the **Rear**) and deleted from the other end (the **Front**).
- **FIFO (First In First Out):** It is called FIFO because the first element added to the queue will be the first one to be removed. This mimics a real-world line (queue) at a ticket counter.

9. Why queue is called ADT?

- **Abstraction:** Similar to a Stack, a Queue is an Abstract Data Type because it specifies a set of operations (Enqueue, Dequeue, Peek) and the behavior (FIFO) without dictating a specific implementation.

- **Implementation Agnostic:** It can be implemented using Arrays, Linked Lists, or even Stacks, but logically, it remains a Queue as long as it adheres to the FIFO contract.

10. What are the applications of priority queue?

- **CPU Scheduling:** Operating systems use them to schedule processes based on priority (e.g., system processes over user processes).
- **Graph Algorithms:** Essential for Dijkstra's Shortest Path Algorithm and Prim's Algorithm for Minimum Spanning Tree.
- **Data Compression:** Used in Huffman Coding to build the Huffman tree.
- **Event Handling:** Simulation systems where events are processed based on timestamp (priority).

11. How can we implement priority queue?

A Priority Queue can be implemented using several data structures, with varying efficiency:

1. **Arrays:** (Ordered or Unordered). Simple but insertion or deletion can be slow ($O(N)$) due to shifting.
2. **Linked Lists:** Elements are inserted in sorted order based on priority.
3. **Heaps (Binary Heap):** The most efficient and common implementation.
    - **Max-Heap:** Root is the maximum priority element.
    - **Min-Heap:** Root is the minimum priority element.
    - Provides $O(\log N)$ complexity for insertion and deletion.

12. What is the limitation of linear queue? How is it solved using circular queue?

- **Limitation (False Overflow):** In a linear queue implemented with an array, if we enqueue elements until the array is full and then dequeue a few, the rear index remains at the end limit. We cannot insert new elements even though there is free space at the beginning (before front).
- **Solution (Circular Queue):** A Circular Queue connects the last position of the array back to the first. When rear reaches the end of the array, the next element is inserted at the 0th index (if it is empty) using the modulo operator.

13. What is the conditions for checking a circular queue using array is empty or not?

- The queue is considered empty if the front pointer is in its initialized invalid state.
- **Condition:** front == -1 (Assuming the implementation initializes front and rear to -1).

14. What is the conditions for checking a circular queue using array is full or not?

- The queue is full if the next position of the rear pointer is the current position of the front pointer.
- **Condition:** (rear + 1) % MAX_SIZE == front

- *(Note: front == 0 && rear == MAX_SIZE - 1 is also a full condition, which is covered by the modulo formula logic).*

15. Write the differences between stack and queue.

Feature	Stack	Queue
Ordering Principle	LIFO (Last In First Out).	FIFO (First In First Out).
Ends Used	Single end (Top) for both insertion and deletion.	Two ends: Rear for insertion, Front for deletion.
Primary Operations	Push and Pop.	Enqueue and Dequeue.
Pointer Names	Top.	Front and Rear.
Visual Metaphor	A pile of books.	A line of people at a counter.

16. Write the operations of priority queue.

- **Insertion (Enqueue):** Adds an item to the queue with an associated priority.
- **Deletion (Dequeue):** Removes the element with the *highest* priority from the queue.
- **Peek:** Returns the element with the highest priority without removing it.

17. In which data structure insertion and deletion of elements can take place from either end? Explain.

- **Data Structure: Deque** (Double Ended Queue).
- **Explanation:** A Deque allows insertion and deletion operations at both the **Front** and the **Rear** ends. It does not enforce strict FIFO or LIFO.
- **Types:**
    - **Input Restricted Deque:** Insertion at one end, deletion at both.
    - **Output Restricted Deque:** Deletion at one end, insertion at both.

18. Differentiate between peek() and pop() functions.

- **Pop():**
    - **Action:** Removes the top element from the stack.
    - **State Change:** The stack size decreases by 1, and the top pointer is decremented.
- **Peek() (or Top()):**
    - **Action:** Returns the value of the top element *without* removing it.
    - **State Change:** The stack size and pointers remain exactly the same.

19. Why are parentheses not required in postfix/prefix expressions?

(Note: This reinforces Q6)

- In Infix expressions like `A + B * C`, parentheses are needed `(A + B) * C` to change the standard precedence.

- In Postfix (A B + C *) and Prefix (* + A B C), the operands are strictly adjacent to their operators in the order they must be calculated. The structure is linear and strictly defined, rendering grouping symbols redundant.

20. Explain how stacks are used in a non-recursive program?

- **Simulating Recursion:** Recursion relies on the system stack to save the state of a function (local variables and return address) before jumping to the next call.
- **Explicit Stack:** To convert a recursive algorithm (like QuickSort or DFS) into a non-recursive (iterative) one, the programmer creates an explicit stack data structure.
- **Usage:** The programmer manually pushes parameters/state onto the stack to simulate a function call and pops them to simulate returning from a function.

21. What do you understand by a multiple stack? How is it useful?

- **Definition:** Multiple Stack usually refers to implementing **two (or more) stacks within a single array**.
- **Implementation:** Typically, Stack A starts at index 0 and grows upwards, while Stack B starts at index MAX-1 and grows downwards.
- **Utility:**
    - **Space Efficiency:** It prevents stack overflow in one stack when there is still free space in the array reserved for the other stack.
    - **Memory Management:** Useful in environments with very limited memory where fixed partitions for separate stacks would be wasteful.

22. Explain the terms infix expression, prefix expression, and postfix expression.

- **Infix Expression:** The operator is placed **between** the operands. This is the standard arithmetic notation.
    - *Example:* A + B
- **Prefix Expression (Polish Notation):** The operator is placed **before** the operands.
    - *Example:* + A B
- **Postfix Expression (Reverse Polish Notation):** The operator is placed **after** the operands.
    - *Example:* A B +

23. Assume that a queue is available for pushing and popping elements. Given an input sequence a, b, c, (c be the first element), give the output sequence of elements if the rightmost element given above is the first to be popped from the queue.

- **Answer:** c, b, a
    - *Reasoning:* A queue follows FIFO (First In First Out). The problem states "c be the first element", implying 'c' was inserted first, followed by 'b', then 'a'. Therefore, 'c' is at the front and must be popped first.

24. Write an algorithm to insert an element into a stack.

Algorithm: PUSH (Stack, MAX, Top, Item)

- **Step 1:** Check for Overflow.
  - IF Top == MAX - 1, THEN Print "Stack Overflow" and Exit.
- **Step 2:** Increment the Top pointer.
  - Top = Top + 1
- **Step 3:** Insert the item.
  - Stack[Top] = Item
- **Step 4:** Exit.

25. Write an algorithm to delete an item from a stack.

Algorithm: POP (Stack, Top)

- **Step 1:** Check for Underflow.
  - IF Top == -1, THEN Print "Stack Underflow" and Exit.
- **Step 2:** Retrieve the item (Optional, if return value is needed).
  - Item = Stack[Top]
- **Step 3:** Decrement the Top pointer.
  - Top = Top - 1
- **Step 4:** Return Item and Exit.

26. Write the insert and delete functions for the queue.

A. Algorithm: ENQUEUE (Queue, MAX, Front, Rear, Item)

1. IF Rear == MAX - 1, Write "Queue Overflow" and Return.
2. IF Front == -1 (Queue is empty), Set Front = 0 and Rear = 0.
3. ELSE, Set Rear = Rear + 1.
4. Set Queue[Rear] = Item.
5. Exit.

B. Algorithm: DEQUEUE (Queue, Front, Rear)

1. IF Front == -1 OR Front > Rear, Write "Queue Underflow" and Return.
2. Set Item = Queue[Front].
3. Set Front = Front + 1.
4. Exit.

27. Write short notes on Priority Queue.

- **Definition:** A Priority Queue is an abstract data type where every element has a "priority" associated with it. Elements are dequeued not strictly by FIFO order, but by priority order.
- **Types:**
  - **Ascending Priority Queue:** The element with the smallest value has the highest priority (e.g., Min-Heap).
  - **Descending Priority Queue:** The element with the largest value has the highest priority (e.g., Max-Heap).
- **Operations:**
  - Insert(item, priority): Add an item.
  - GetHighestPriority(): Peek at the highest priority item.
  - DeleteHighestPriority(): Remove the highest priority item.

- **Implementation:** Can be implemented using Arrays, Linked Lists, or Heaps. Heaps are the most efficient implementation, offering $O(\log n)$ for insertion and deletion.

28. Write an algorithm to convert a decimal number to binary using a stack.
Algorithm: Decimal_To_Binary(Number)

1. Initialize an empty Stack S.
2. **While** Number > 0:
    - Calculate remainder: rem = Number % 2.
    - Push rem onto Stack S.
    - Update Number: Number = Number / 2 (integer division).
3. **While** Stack S is not empty:
    - Pop element x from S.
    - Print x.
4. **End.** (The printed sequence is the binary representation).

29. Write an algorithm to print all the prime factors of a number in non-increasing order using a stack.
Note: Standard factorization generates factors in non-decreasing order (e.g., 2, 3, 5). Using a stack reverses this to non-increasing (e.g., 5, 3, 2).
Algorithm: Prime_Factors_Descending(N)

1. Initialize an empty Stack S.
2. Set divisor = 2.
3. **While** N > 1:
    - **While** N % divisor == 0:
        - Push divisor onto Stack S.
        - Set N = N / divisor.
    - Increment divisor.
4. **While** Stack S is not empty:[1]
    - Pop element x from S.[2]
    - Print x.[3]
5. **End**

30. Write the insert function for the queue
(This refers to the standard linear queue)6
Algorithm: Insert_Queue(Q, MAX, Rear, Item)7

1. **Ch[8]eck Overflow:** IF Rear == MAX - 1, Print "Overflow" and Exit.
2. **Handle Empty Queue:** IF Front == -1, Set Front = 0.
3. **Increment Rear:** Set Rear = Rear + 1.
4. **Insert:** Set Q[Rear] = Item.
5. **Exit.**

31. Write the delete functions for the queue.
Algorithm: Delete_Queue(Q, Front, Rear)

1. **Check Underflow:** IF Front == -1 OR Front > Rear, Print "Underflow" and Exit.

2. **Retrieve Data:** Set Item = Q[Front].
3. **Increment Front:** Set Front = Front + 1.
4. **Exit.**

32. Write Q-insert algorithm for circular queue.

Algorithm: Circular_Enqueue(Q, MAX, Front, Rear, Item)

1. **Check Full:** IF (Rear + 1) % MAX == Front, Print "Queue Full" and Exit.
2. **Check Empty:** IF Front == -1:
    - Set Front = 0.
    - Set Rear = 0.
3. **Else:**
    - Set Rear = (Rear + 1) % MAX.
4. **Insert:** Set Q[Rear] = Item.
5. **Exit.**

33. Write an algorithm to convert an infix expression to its postfix form using stack.

Algorithm: Infix_To_Postfix(Expression)

1. Initialize an empty Stack S and an empty Output List L.
2. Scan the Expression from left to right.
3. **If** character x is an operand (variable/number):
    - Add x to L.
4. **If** character x is (:
    - Push (onto S.
5. **If** character x is):
    - Pop from S and add to L until (is encountered.
    - Discard the (.
6. **If** character x is an operator (+, -, *, /):
    - **While** S is not empty AND Precedence(Top of S) ≥ Precedence(x):
        - Pop from S and add to L.
    - Push x onto S.
7. **While** S is not empty:
    - Pop from S and add to L.
8. **End:** L contains the Postfix expression.

34. Write an algorithm to evaluate the postfix expression using stack.

Algorithm: Evaluate_Postfix(Expression)

1. Initialize an empty Stack S.
2. Scan the Expression from left to right.
3. **If** character x is an operand:
    - Push x (value) onto S.
4. **If** character x is an operator:
    - Pop element A from S (Top element).
    - Pop element B from S (Next element).

- Calculate Result = B operator A (Note order: Second popped op First popped).
- Push Result onto S.

5. **End:** Pop the final value from S. This is the answer.

35. In a circular queue implemented using an array of size 7, the initial values of both front and rear are set to 0. Determine the final values of front and rear...
Final Values: Front = 5, Rear = 4

- **Logic:**
  - **Formula:** NewValue = (OldValue + Steps) % Size
  - Rear (Insertions): Total insertions = 5 + 6 = 11.
    Rear = (0 + 11) \% 7 = 4
  - Front (Deletions): Total deletions = 2 + 3 = 5. (Peeks do not move pointers).
    Front = (0 + 5) % 7 = 5

36. Assume that the operators +, -, * are left associative and ^ is right associative... What will be the postfix expression corresponding to the infix expression a+b*c-d^e^f ?
Answer: a b c * + d e f ^ ^ -

- **Step-by-step:**
  1. Exponentiation (Right Associative): d ^ e ^ f -> d ^ (e ^ f) -> d e f ^ ^
  2. Multiplication: b * c -> b c *
  3. Addition (Left Associative): a + (b c *) -> a b c * +
  4. Subtraction: (a b c * +) - (d e f ^ ^) -> a b c * + d e f ^ ^ -

37. Pick out the equivalent postfix expression for this infix expression: X=((A+B)C-(D-E)^(F+G))
Answer: A B + C * D E - F G + ^ -

38. Convert the following infix expressions to their postfix equivalents:
(Note: Questions 4 and 5 are duplicates in your list. Solutions provided below).

- **(a) A - B + C**
  - Answer: A B - C +
- **(b) A * B + C / D**
  - Answer: A B * C D / +
- **(c) (A - B) + C * D / E - C**
  - Answer: A B - C D * E / + C -
- **(d) (A * B) + (C / D) - (D + E)**
  - Answer: A B * C D / + D E + -
- **(e) ((A - B) + D / ((E + F) * G))**
  - Answer: A B - D E F + G * / +
- **(f) (A - 2 * (B + C) / D * E) + F**
  - Answer: A 2 B C + * D / E * - F +
- **(g) 14 / 7 * 3 - 4 + 9 / 2**

- Answer: 14 7 / 3 * 4 - 9 2 / +

39. Find the infix equivalents of the following postfix equivalents

- **(a) A B + C * D –**
  - Answer: ((A + B) * C) - D
- **(b) A B C * + D –**
  - Answer: (A + (B * C)) - D

40. Give the infix expression of the following prefix expressions.

- **(a) * - + A B C D**
  - Answer: ((A + B) - C) * D
- **(b) + - a * B C D**
  - Answer: (a - (B * C)) + D

41. Convert the expression given below into its corresponding postfix expression and then evaluate it. Also write a program to evaluate a postfix expression.

Expression: 10 + ((7 - 5) + 10)/2
Postfix: 10 7 5 - 10 + 2 / +
Evaluation:

1. 7 - 5 = 2
2. 2 + 10 = 12
3. 12 / 2 = 6
4. 10 + 6 = 16

Result: 16

Algorithm (Program Logic):

1. Create a stack to store operands.
2. Scan the postfix expression from left to right.
3. If the character is a number, PUSH it onto the stack.
4. If the character is an operator (+, -, *, /):
   - POP two elements (Top is Op2, Next is Op1).
   - Compute val = Op1 operator Op2.
   - PUSH val back onto the stack.
5. After scanning, the final result is the only element remaining in the stack.

42. Write a function that accepts two stacks. Copy the contents of first stack in the second stack.

Note: To preserve order, we need a temporary stack.

Algorithm:

1. Initialize Stack1 (Source), Stack2 (Destination), and TempStack.
2. **While** Stack1 is not empty:
   - Pop element X from Stack1.
   - Push X onto TempStack.
3. **While** TempStack is not empty:
   - Pop element X from TempStack.
   - Push X onto Stack2.

4. **End:** Stack2 now contains Stack1's elements in the original order.

43. Draw the stack structure in each case when the following operations are performed on an empty stack.

- **(a) Add A, B, C, D, E, F:** Stack is [A, B, C, D, E, F] (F is Top).
- **(b) Delete two letters:** F and E removed. Stack is [A, B, C, D].
- **(c) Add G:** Stack is [A, B, C, D, G].
- **(d) Add H:** Stack is [A, B, C, D, G, H].
- **(e) Delete four letters:** H, G, D, C removed. Stack is [A, B].
- **(f) Add I:** Stack is [A, B, I].

44. A single array A[1..MAXSIZE] is used to implement two stacks... What is the condition for "stack full"?

- **Setup:** Stack 1 starts at index 1 and grows up. Stack 2 starts at MAXSIZE and grows down.
- **Condition:** Top1 == Top2 - 1
- **Explanation:** This indicates that the pointers have met and there is no free space left in the array between them.

45. Describe the stack as the following operations take place (Initial: A, C, D, F, K):

- **Initial Top:** K.
- (a) POP: K removed. Top = F.
- (b) POP: F removed. Top = D.
- (c) PUSH(L): Stack = A, C, D, L.
- (d) PUSH(P): Stack = A, C, D, L, P.
- (e) POP: P removed. Top = L.
- (f) PUSH(R): Stack = A, C, D, L, R.
- (g) PUSH(S): Stack = A, C, D, L, R, S.
- (h) POP: S removed. Top = R.
- **Final Stack:** A, C, D, L, R.

46. Describe the following operations on queue (Size 5. F=2, R=3, Q: _, P, Q, _, _):

- *Current Indices:* P at 2, Q at 3.
- **(a) R is added:** Rear moves to 4. Queue: [_, _, P, Q, R].
- **(b) Two letters removed:** P (index 2) and Q (index 3) are removed. Front moves to 4.
- **(c) S, T, U are added:**
  - Add S: Rear wraps to 0. Queue: [S, _, _, _, R].
  - Add T: Rear moves to 1. Queue: [S, T, _, _, R].
  - Add U: Rear moves to 2. Queue: [S, T, U, _, R].
  - *Result:* Queue contains R, S, T, U. Front at 4, Rear at 2.

47. Write logic for reverse the order of elements on a stack S.

i) Using two additional stacks:

1. Pop all elements from S and push to Stack1 (Order reverses).
2. Pop all elements from Stack1 and push to Stack2 (Order restores to original).
3. Pop all elements from Stack2 and push to S (Order reverses again). **Final S is reversed.**

ii) Using one additional queue:

1. Pop all elements from S and Enqueue them into Queue (Order reverses).
2. Dequeue all elements from Queue and Push them back into S. **Final S is reversed.**

48. If n=5 it means 1, 2,3,4,5 then what is the printed sequence?

(Assuming standard behavior: Pushing all numbers 1 through 5, and then Popping them to empty the stack)

Output: 5, 4, 3, 2, 1

- **Reasoning:** Stack is LIFO. 1 enters first, 5 enters last. Therefore, 5 is popped first.

49. Convert A + (B * C - (D/E ^ F) * G) * H into its postfix form. Evaluate: 5 6 2 + * 12 4 / - using stack.

Part 1: Postfix Conversion

A B C * D E F ^ / G * - H * +

Part 2: Evaluation (5 6 2 + * 12 4 / -)

1. Push 5.
2. Push 6, Push 2.
3. Operate +: 6 + 2 = 8. (Stack: 5, 8).
4. Operate *: 5 * 8 = 40. (Stack: 40).
5. Push 12, Push 4.
6. Operate /: 12 / 4 = 3. (Stack: 40, 3).
7. Operate -: 40 - 3 = 37.

Final Result: 37

50. What will be the top two elements of the stack after the first * is evaluated?

Expression: 8 2 3 ^ / 2 3 * + 5 1 * -

- **Trace:**
    1. Push 8, 2, 3.
    2. ^ (Exponent): 2^3 = 8. Stack: [8, 8].
    3. / (Divide): 8 / 8 = 1. Stack: [1].
    4. Push 2, Push 3. Stack: [1, 2, 3].
    5. * (Multiply): 2 * 3 = 6. Stack: [1, 6].
- **Answer:** The top two elements are **6** (Top) and **1** (Below it).

51. Convert A$B * C - D + E/F/(G + G) into prefix.

(Assuming $ represents exponentiation)

Answer: + - * $ A B C D / / E F + G G

52. What is the condition for stack full (Two stacks, one array)?

- **Answer:** top1 == top2 - 1
- **Explanation:** This means the top pointer of the first stack is adjacent to the top pointer of the second stack; there is no memory cell left between them.

53. Analyze this code and evaluate the uses of this code?

```
void insert (Q, x) {
      push (S1, x);
}
void delete (Q){
      if (stack-empty (S2)) then
      if (stack-empty (S1)) then
      {
            print ( "Q is empty" );
            return;
      }
      else
            while (! (stack-empty (S1))) then
            {
                  x = pop (S1);
                  push (S2, x);
            }
            x = pop (S2);
}
```

Analysis:

- **Purpose:** This code implements a **Queue using two Stacks** (S1 and S2).
- **Logic:**
  - **Insert (enqueue):** Elements are always pushed onto Stack 1 (S1). This operation is $O(1)$.
  - **Delete (dequeue):**
    - The function checks Stack 2 (S2). If S2 has elements, it pops the top element (FIFO order preserved).
    - If S2 is empty, it transfers **all** elements from S1 to S2 by popping from S1 and pushing to S2. This reverses the order of elements (LIFO to FIFO).
    - After the transfer, the top of S2 is popped.
- **Complexity:**
  - Insertion: $O(1)$.
  - Deletion: Amortized $O(1)$, though a single worst-case deletion involves $O(n)$ work (moving all elements).

Uses/Applications:

- **Restricted Environments:** Used in systems or functional programming languages where Stack is a primitive data structure, but Queue is not natively implemented.
- **Undo/Redo Logic:** Can be adapted for complex undo operations where operations are batched (S1) and then processed linearly (S2).

- **Interview/Academic:** A standard problem to demonstrate understanding of LIFO vs. FIFO transformations.

54. Let n insert and m (≤ n) delete operations be performed in an arbitrary order on an empty queue Q. let x and y be the number of push and pop operations performed respectively in the process. Which one of the following is true for all m and n?

(A) $n + m \leq x < 2n$ and $2m \leq y \leq n + m$
(B) $n + m \leq x < 2n$ and $2m \leq y \leq 2n$
(C) $2m \leq x < 2n$ and $2m \leq y \leq n + m$
(D) $2m \leq x < 2n$ and $2m \leq y \leq 2n$

55. Can you implement a stack using queue, justify your answer.
Answer: Yes, a Stack can be implemented using Queues.
Justification:
A Stack follows LIFO (Last In First Out), while a Queue follows FIFO. To achieve LIFO using FIFO, we must manipulate the order during either the push or pop operation.

- **Method 1 (Two Queues):**
  - Take two queues, Q1 and Q2.
  - **Push operation (O(n)):** Enqueue the new element to Q2. Then, dequeue all elements from Q1 and enqueue them to Q2. Swap the names of Q1 and Q2.
  - **Pop operation (O(1)):** Simply dequeue from Q1.
- **Method 2 (Single Queue):**
  - **Push operation (O(n)):** Enqueue the element, then dequeue and re-enqueue the existing elements (size - 1 times) so the new element moves to the front.

56. Can you implement a queue using stack, justify your answer.
Answer: Yes, a Queue can be implemented using Stacks.
Justification:
This is the exact logic analysed in Question 1.

- **Requirement:** Two Stacks (S1 and S2).
- **Enqueue (O(1)):** Always push to S1.
- **Dequeue (Amortized O(1)):**
  - Pop from S2.
  - If S2 is empty, pop everything from S1 and push it into S2 (reversing the order), then pop from S2.
- **Reasoning:** Stacks reverse order. Reversing the order twice (Push to S1 -> Pop S1/Push S2 -> Pop S2) restores the original sequence, achieving FIFO behaviour.

57. While writing a C/C++ code, the problem faced by the programmers is to find if the parenthesis is balanced or not. Write an algorithm to check if the

parenthesis in C code are balanced. Initially your code should work for balanced { and } braces.

Algorithm: Check_Balanced_Parentheses(Expression)

1. Initialize an empty Stack S.
2. Traverse the Expression string from left to right, character by character (char).
3. **If** char is an opening brace {:
    - Push { onto Stack S.
4. **Else If** char is a closing brace }:
    - **Check Empty:** If Stack S is empty, return **Unbalanced** (Missing opening brace).
    - **Pop:** Pop the top element from Stack S.
    - *(Note: Since we are checking only { and }, we assume the popped element matches. For multiple types () {} [], we would verify the popped character is the corresponding opening pair.)*
5. **End of Traversal:**
    - If Stack S is **Empty**: Return **Balanced**.
    - If Stack S is **Not Empty**: Return **Unbalanced** (Missing closing brace).

Recursion

1. What is recursion? Explain with an example.

- **Definition:** Recursion is a programming technique where a function calls itself directly or indirectly to solve a problem. It breaks a complex problem into smaller instances of the same problem.
- **Components:**
  - **Base Case:** The condition that stops the recursion.
  - **Recursive Case:** The part where the function calls itself.
- **Example (Factorial):** n! = n \times (n-1)!. To calculate fact(3), the function calls 3 * fact(2), which calls 2 * fact(1).

2. What is recursion tree?

- **Definition:** A recursion tree is a visual representation (graph) used to illustrate the flow of recursive function calls.
- **Structure:**
  - **Root:** Represents the initial function call.
  - **Branches:** Represent the recursive calls made by the parent node.
  - **Nodes:** Represent the parameters passed to each call.
  - **Leaves:** Represent the base cases where recursion stops.
- **Usage:** It is highly useful for analyzing the time complexity of recursive algorithms (e.g., Merge Sort, Fibonacci).

3. Write down the recursive definition for generation of the Fibonacci sequence.

The Fibonacci sequence is defined mathematically as:

$$F(n) = \begin{cases} 0 & if\ n = 0 \\ 1 & if\ n = 1 \\ F(n-1) + F(n-2) & if\ n > 1 \end{cases}$$

4. What is tail recursion?

- **Definition:** Tail recursion is a special form of recursion where the recursive call is the **very last operation** performed by the function.
- **Significance:** No calculation is performed on the returned value of the recursive call.
- **Optimization:** Compilers can optimize tail recursion (Tail Call Optimization) by reusing the current stack frame instead of creating a new one, preventing Stack Overflow.

5. Write short notes on Tail Recursion and Tower of Hanoi Problem.

- **Tail Recursion:**
  - Occurs when the recursive call is the final statement.
  - *Example:* return recursive_call(n-1); (Tail) vs return n * recursive_call(n-1); (Non-Tail).

- It is memory efficient ($O(1)$ auxiliary space) compared to standard recursion ($O(N)$ stack space).

- **Tower of Hanoi:**
  - A mathematical puzzle with three rods and n disks of different sizes.
  - **Objective:** Move the entire stack from the source rod to the destination rod using an auxiliary rod.
  - **Rules:** Only one disk moves at a time, and a larger disk cannot be placed on top of a smaller disk.
  - It is a classic example of standard recursion with exponential complexity.

6. Calculate the time complexity of Tower of Hanoi problem.

- Recurrence Relation: Let T(n) be the time to move n disks.

$$T(n) = 2T(n-1) + 1$$

(Move n-1 disks to aux, move largest to dest, move n-1 from aux to dest).

- Solving:

$$T(n) \approx 2 \times 2^{n-1} \approx 2^n$$

- **Complexity:** $O(2^n)$ (Exponential Time).

7. Draw the recursion tree for number of disks equals to 3.

(Structure description for exam drawing):

- **Root:** Move(3, A, C, B)
- **Level 1:** Calls Move(2, A, B, C) (Left) and Move(2, B, C, A) (Right).
- **Level 2:** Each Level 1 node spawns two calls for Move(1, ...) and a print step.
- **Total Calls:** $2^3 - 1 = 7$ nodes in the tree.

8. Differentiate between an iterative function and a recursive function.

Feature	Recursive Function	Iterative Function
Definition	Function calls itself.	Uses loops (for, while) to repeat.
Memory	High overhead (uses Stack memory).	Low overhead (uses CPU registers/variables).
Speed	Generally slower due to context switching.	Generally faster.
Code Size	Concise and cleaner for complex logic (e.g., trees).	Can become complex and lengthy.
Risk	Risk of Stack Overflow.	Risk of Infinite Loop.

9. Write the recursive function to find the factorial of a number.

```
int factorial(int n) {
    if (n == 0) // Base Case
        return 1;
    else        // Recursive Case
        return n * factorial(n - 1);
}
```

10. Write the recursive function to find nth Fibonacci number.

```
int fib(int n) {
    if (n == 0) return 0; // Base Case 1
    if (n == 1) return 1; // Base Case 2
    return fib(n - 1) + fib(n - 2); // Recursive Case
}
```

11. Explain the execution order of factorial (5) using stack.

When fact(5) is called, the system pushes frames onto the stack until the base case is reached.

I. Pushing Phase (Winding):

1. fact(5) calls fact(4) -> Push 5
2. fact(4) calls fact(3) -> Push 4
3. fact(3) calls fact(2) -> Push 3
4. fact(2) calls fact(1) -> Push 2
5. fact(1) calls fact(0) -> Push 1
6. fact(0) returns 1 (Base Case).

II. Popping Phase (Unwinding):

1. fact(1) returns 1 * 1 = 1
2. fact(2) returns 2 * 1 = 2
3. fact(3) returns 3 * 2 = 6
4. fact(4) returns 4 * 6 = 24
5. fact(5) returns 5 * 24 = 120

12. Write the recursive function for the problem of Tower of Hanoi problem.

```
void towerOfHanoi(int n, char from_rod, char to_rod, char aux_rod) {
    if (n == 1) {
        printf("\n Move disk 1 from rod %c to rod %c", from_rod, to_rod);
        return;
    }
    // Move n-1 disks from A to B
    towerOfHanoi(n - 1, from_rod, aux_rod, to_rod);
    // Move largest disk from A to C
    printf("\n Move disk %d from rod %c to rod %c", n, from_rod, to_rod);
    // Move n-1 disks from B to C
    towerOfHanoi(n - 1, aux_rod, to_rod, from_rod);
}
```

13. Assuming Fib (n) as a recursive function draw a recursive tree for Fib (6).

(Note: Fib(6) creates a large tree. The structure is binary).

- **Root:** Fib(6)
- **Level 1:** Fib(5) and Fib(4)
- **Level 2:**
  - Fib(5) splits into Fib(4) and Fib(3)
  - Fib(4) splits into Fib(3) and Fib(2)
- The tree continues expanding until all leaf nodes are Fib(1) or Fib(0).
- Note the **Overlapping Subproblems**: Fib(4) and Fib(3) are calculated multiple times.

14. Write the function for tail recursion to find the factorial of a number.

- To make it tail recursive, we use an **accumulator** (acc) to carry the result.

```
int factTail(int n, int acc) {
    if (n == 0)
        return acc;
    return factTail(n - 1, n * acc); // Tail call
}
// Usage: factTail(5, 1);
```

15. Write the function for tail recursion to find nth Fibonacci number.

- We use two accumulators (a and b) to store the previous two numbers.

```
int fibTail(int n, int a, int b) {
    if (n == 0) return a;
    if (n == 1) return b;
    return fibTail(n - 1, b, a + b); // Tail call
}
// Usage: fibTail(n, 0, 1);
```

Linked List

1. Make a comparison between a linked list and a linear array. Which one will you prefer to use and when?

Feature	Linear Array	Linked List
Memory Allocation	Static (compile-time) or Dynamic (contiguous).	Dynamic (runtime). Elements are non-contiguous.
Access Time	O(1) Random Access (using index).	O(n) Sequential Access (must traverse from head).
Insertion/Deletion	Costly (O(n)) due to shifting elements.	Efficient (O(1)) if pointer location is known.
Memory Efficiency	efficient for data, but may waste space if declared large.	Requires extra memory for pointers (overhead).

Preference:

- **Use Array:** When frequent access to elements by index is required and the number of elements is known in advance (e.g., lookup tables).
- **Use Linked List:** When the data size is dynamic (unpredictable) and there are frequent insertions/deletions of data (e.g., implementing a queue or undo functionality).

2. Why is a doubly linked list more useful than a singly linked list?

- **Bidirectional Traversal:** Nodes contain pointers to both next and prev. You can traverse forward and backward, which is impossible in a singly linked list.
- **Efficient Deletion:** To delete a node in a singly linked list, you need the address of the *previous* node (requiring traversal). In a doubly linked list, given a pointer to a node, you can access its predecessor immediately via the prev pointer to perform deletion in O(1) time.

3. Give the advantages and uses of a circular linked list.

- **Advantages:**
  - Any node can be a starting point to traverse the whole list.
  - Useful for implementing a Queue, as we can maintain a pointer to the last inserted node, allowing access to the front and rear in constant time.
- **Uses:**
  - Round-robin scheduling in Operating Systems (assigning CPU time slots).
  - Implementation of advanced data structures like Fibonacci Heaps.
  - Multiplayer gaming environments (passing turns in a circle).

4. Specify the use of a header node in a header linked list.

- **Definition:** A header node is a special dummy node at the beginning of the list that contains no valid data (or meta-data like list count).

- **Use:** It simplifies insertion and deletion algorithms. With a header node, the "first" data node is treated exactly like any other node (it has a predecessor—the header). This eliminates the need for special if (head == NULL) logic or updating the external head pointer.

5. Give the linked representation of the following polynomial: 7x^5 + 3x^4 - 8x^3 + 3x^2 + 11x - 4

The polynomial is represented by a linked list where each node has three fields: Coefficient, Exponent, and Next.

Representation:[Coef: 7 | Exp: 5 | Next] -> [3 | 4 | Next] -> [-8 | 3 | Next] -> [3 | 2 | Next] -> [11 | 1 | Next] -> [-4 | 0 | NULL]

6. Explain the difference between a circular linked list and a singly linked list.

- **Structure:** In a Singly Linked List, the next pointer of the last node points to NULL. In a Circular Linked List, the next pointer of the last node points back to the Head (first node).
- **Termination:** A singly linked list traversal terminates when NULL is reached. A circular linked list traversal needs a check to see if the pointer has returned to the Head, otherwise, it results in an infinite loop.

7. Form a linked list to store students' details.

Structure Definition (C Language):

```
struct Student {
    int roll_no;
    char name[50];
    float marks;
    struct Student* next;
};
struct Student* head = NULL;
```

Concept: Each node stores the student's roll number, name, and marks. The list is formed by allocating memory dynamically using malloc for each student and linking them.

8. Use the linked list of the above question to insert the record of a new student in the list.

Algorithm (Insert at End):

1. Create a newNode and populate roll_no, name, marks. Set newNode->next = NULL.
2. **If** head == NULL, set head = newNode.
3. **Else**:
    - Initialize temp = head.
    - Traverse while temp->next != NULL.
    - Set temp->next = newNode.

9. Delete the record of a student with a specified roll number from the list mentioned in question 7.

Algorithm:

1. **If** head is empty, return.
2. **If** head->roll_no == target:
    - temp = head; head = head->next; free(temp); return.
3. Initialize prev = head, current = head->next.
4. **While** current != NULL AND current->roll_no != target:
    - Move prev and current forward.
5. **If** current == NULL, record not found.
6. **Else**:
    - prev->next = current->next.
    - free(current).

10. Create two linked lists- one which stores upper case characters and the other that stores lower case characters.

Logic:

1. Initialize two headers: headUpper = NULL, headLower = NULL.
2. Traverse the original list containing characters.
3. For each node:
    - **If** (data >= 'A' AND data <= 'Z'): Insert into headUpper list.
    - **If** (data >= 'a' AND data <= 'z'): Insert into headLower list.
4. Standard insertion logic applies for creating the new lists.

11. Create a linked list which stores names of the employees. Then sort these names and re-display...

Approach:

1. **Creation:** Define a struct with char name[50]. Insert nodes.
2. **Sorting (Bubble Sort on List):**
    - Use nested loops. The outer loop controls the passes, the inner loop traverses the list.
    - Compare adjacent strings using strcmp(node1->name, node2->name).
    - If order is incorrect (> 0), swap the **data** (names) of the nodes (swapping data is easier than swapping pointers).
3. **Display:** Traverse the sorted list and print names.

12. Is linked list can be considered as a linear data structure? Explain.

Answer: Yes.

- **Explanation:** Although the nodes of a linked list are stored non-contiguously in memory (scattered addresses), the **logical** relationship between the elements is linear. Each element points to exactly one successor (except the last), forming a sequential "line" of data.

13. If you are using C language to implement the heterogeneous linked list, what pointer type will you use?

Answer: void * (Void Pointer).

- **Explanation:** A heterogeneous linked list stores data of different types (int, float, char). Since C is statically typed, a standard pointer cannot hold different types. A void * is a generic pointer that can point to any data type. The node structure would contain a void *data and likely a type_tag (enum) to indicate what kind of data is currently stored to cast it correctly during retrieval.

14. What is self-referential structure? Explain with an example.

- **Definition:** A self-referential structure is a structure that contains a pointer to a variable of the **same structure type**. This is the building block of Linked Lists and Trees.
- **Example:**

```
struct Node {
    int data;
    struct Node *next; // Pointer to the same structure type
};
```

15. Write a function to remove the first node from the list and insert it at end...

```
void moveFirstToLast(struct Node** head_ref) {
    if (*head_ref == NULL || (*head_ref)->next == NULL)
        return; // List is empty or has only 1 node

    struct Node* first = *head_ref;
    struct Node* last = *head_ref;

    // Move 'first' to become the new head's next
    *head_ref = first->next;

    // Traverse to the end
    while (last->next != NULL) {
        last = last->next;
    }

    // Attach the original first node to the end
    last->next = first;
    first->next = NULL;
}
```

16. Which of the following statements are true in the case of doubly linked list?
Answer: Statement **(ii)** is true. "We can traverse in both the directions".

- **Explanation:** A Doubly Linked List node has a prev pointer. From any specific node, you can navigate to node->next (forward) and node->prev (backward).
- Statement (i) is false because typically the last node is not connected to the first (unless circular).

17. What does the following function do for a given Linked List with first node as head? (fun1)

```
void fun1(structnode* head){
      if(head == NULL)
            return;
```

```
      fun1(head->next);
      printf("%d ", head->data);
}
```

Answer: It prints the elements of the linked list in **Reverse Order**.

- **Explanation:** The function is recursive. It calls fun1(head->next) *before* the printf statement. This means the recursion dives deep until the end of the list (NULL). As the stack unwinds (returns), it executes the print statement for the last node, then the second to last, and so on, back to the head.

18. Can we perform insertion sort in linked lists? Explain.

Answer: Yes.

- **Explanation:** Insertion sort is actually quite suitable for linked lists. Unlike arrays, inserting an element into the sorted portion of the list does not require shifting elements; it only requires updating pointers. We create a "Sorted" list (initially empty) and iterate through the input list, picking nodes one by one and inserting them into the correct position in the "Sorted" list.

19. The following function reverse () is supposed to reverse a singly linked list. There is one line missing at the end of the function.

```
/* Link list node */
Struct node{
      Int data;
      Struct node* next;
};
/* head_ref is a double pointer which points to head (or start) pointer of
linked list */
static void reverse(struct node** head_ref)
{
      struct node* prev = NULL;
      struct node* current = *head_ref;
      struct node* next;
      while(current != NULL)
      {
            next = current->next;
            current->next = prev;
            prev = current;
            current = next;
      }
      /*ADD A STATEMENT HERE*/
}
```

What should be added in place of /*ADD A STATEMENT HERE*/, so that the function correctly reverses a linked list ?

Answer: *head_ref = prev;

- **Explanation:** At the end of the while loop, current becomes NULL, and prev points to the last valid node of the original list. In a reversed list, the original last node becomes the new head. We must update the head pointer to point to prev.

20. Consider the function f defined below.

```
struct item
{
      int data;
      struct item * next;
};
int f(struct item *p)
{
      return (
            (p == NULL) ||
            (p->next == NULL) ||
            (( P->data <= p->next->data) && f(p->next))
      );

}
```

For a given linked list p, when the function f returns 1?

Answer: It returns **1** if the linked list is **Sorted (Non-Decreasing Order)** or if the list is empty/has one node.

- **Explanation:**
  - Base cases: Returns 1 if p is NULL or p->next is NULL.
  - Recursive step: It checks if current_data <= next_data. If true, it recursively checks the rest of the list. If it encounters any pair where current > next, the && condition fails and it returns 0.

21. Is it possible to find a loop in a Linked list? Give reason to your answer.

Answer: Yes.

- **Reason:** We can use **Floyd's Cycle-Finding Algorithm** (Tortoise and Hare). We initialize two pointers, slow (moves 1 step) and fast (moves 2 steps). If there is a loop, the fast pointer will eventually enter the loop and "lap" the slow pointer (i.e., slow == fast). If fast reaches NULL, there is no loop.

22. How would you sort the elements in a single linked list?

The most efficient sorting algorithm for a Linked List is **Merge Sort** ($O(n \log n)$).

1. **Divide:** Find the middle of the list (using slow/fast pointers) and split into two halves.
2. **Conquer:** Recursively sort the two halves.
3. **Combine:** Merge the two sorted halves into a single sorted list.*Note: QuickSort is generally preferred for Arrays, Merge Sort for Linked Lists because Merge Sort doesn't require random access.*

23. Linked-list has odd and even numbers. Write an algorithm to find them and position of them.

Algorithm:

1. Initialize count = 1 and ptr = head.
2. **While** ptr != NULL:
   - **If** ptr->data % 2 == 0: Print "Even number [data] at position [count]".
   - **Else**: Print "Odd number [data] at position [count]".

- ptr = ptr->next.
- count++.

24. Which operations are more effective in linked list than arrays? And why?
i) Insertion
ii) Deletion
iii) Traversal
Answer: i) Insertion and **ii) Deletion**.

- **Why:** In an array, inserting or deleting an element (specifically at the beginning or middle) requires shifting all subsequent elements, taking O(n) time. In a Linked List, these operations only involve changing pointers (once the location is reached), which is O(1) work.
- **Note:** Traversal is *not* more effective in LL (it is sequential O(n) vs Random Access O(1) in Arrays).

25. Can we do a Binary search on a linked list? Give reason to your answer.
Answer: It is possible but **highly inefficient** and generally not done.

- **Reason:** Binary search requires random access to the middle element (O(1)). In a Linked List, finding the middle element requires traversing half the list (O(n/2)). This increases the time complexity to O(n), negating the O(log n) advantage of Binary Search.

26. If you implement stack or queue using linked list, then what is the full conditions?
Answer: Heap Memory Exhaustion.

- **Explanation:** A Linked List implementation is dynamic and only grows until the system runs out of memory. The "Full" condition occurs when malloc returns NULL.

```
struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
if (temp == NULL) { /* Stack/Queue is Full */ }
```

27. Is Circular linked list infinite? Explain.
Answer: No.

- **Explanation:** A circular linked list contains a finite number of nodes. It is called "circular" because the traversal path is a closed loop (infinite traversal is possible without hitting NULL), but the data structure itself occupies finite memory.

28. Application of single and double linked list.

- **Singly Linked List:** Used for Symbol Tables in compilers, implementation of Stacks and Queues, and chaining in Hash Tables (to handle collisions).
- **Doubly Linked List:** Used in navigation systems (Browser Back/Forward buttons), Undo/Redo functionality in editors, and music players (Previous/Next song).

29. Who is responsible to dynamically allocate the memory for creation of a node in a single linked list?

Answer: The **Programmer**.

- The programmer must explicitly call memory allocation functions (like malloc in C or new in C++) to create nodes and must also free the memory (free or delete) to prevent memory leaks.

30. Write the recursive definition of Tower of Hanoi. Then find the running time in terms of recurrence relation.

- **Definition:** To move n disks from Source (A) to Destination (C) using Auxiliary (B):
  1. Move n-1 disks from A to B.
  2. Move the nth (largest) disk from A to C.
  3. Move n-1 disks from B to C.
- **Recurrence Relation:** $T(n) = 2T(n-1) + 1$
- **Running Time:** $O(2^n)$ (Exponential).

31. Explain head and tail recursion with an example.

- **Head Recursion:** The recursive call is made *before* other processing in the function (processing happens as recursion returns).
  - *Example:* fun(n) { if(n>0) { fun(n-1); print(n); } } (Prints 1 2 3).
- **Tail Recursion:** The recursive call is the *last* statement executed (processing happens before the call).
  - *Example:* fun(n) { if(n>0) { print(n); fun(n-1); } } (Prints 3 2 1).

32. Draw the recursive tree of Tower of Hanoi.

(Descriptive representation for exam purposes - Assuming n=3)

- **Root:** Move(3, A, C, B)
  - **Left Child:** Move(2, A, B, C)
  - **Middle Action:** Print "Move disk 3 A->C"
  - **Right Child:** Move(2, B, C, A)*(The tree expands further for n=2 and n=1, creating a binary structure).*

33. The following recursive function in C is a solution to the Towers of Hanoi problem.

```
void move (int n, char A, char B, char C)
{
      if (.....................................)
      {
            move (.....................................);
            printf("Move disk %d from pole %c to pole %c\n",n, A,C);
            move (.....................................);
      }
}
```

Fill in the dotted parts of the solution.

```
void move (int n, char A, char B, char C)
{
    if (n > 0)  // Base condition: n must be greater than 0
    {
        // Move n-1 disks from Source (A) to Aux (B) using Dest (C) as
buffer
        move (n - 1, A, C, B);

        printf("Move disk %d from pole %c to pole %c\n", n, A, C);

        // Move n-1 disks from Aux (B) to Dest (C) using Source (A) as
buffer
        move (n - 1, B, A, C);
    }
}
```

34. How to read a singly linked list in backward?

- **Method:** Since a singly linked list only has forward pointers, we cannot traverse it backward iteratively without modifying the structure (like reversing it first).
- **Solution (Recursion):** The most standard academic approach is using **Recursion**. We traverse to the end of the list using the recursion stack, and print the data while the function returns (unwinds).
- **Code Snippet:**

```
void printReverse(struct Node* head) {
    if (head == NULL)
        return;
    printReverse(head->next); // Recursive call
    printf("%d ", head->data); // Print on return
}
```

35. Write a C/C++ function for insertion of a data item after a specified data item into a linear linked list.

```
void insertAfterKey(struct Node* head, int key, int newData) {
    struct Node* temp = head;
    // 1. Search for the key
    while (temp != NULL && temp->data != key) {
        temp = temp->next;
    }
    // 2. If key not found
    if (temp == NULL) {
        printf("Key not found.\n");
        return;
    }
    // 3. Create new node
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = newData;

    // 4. Link new node
    newNode->next = temp->next;
    temp->next = newNode;
}
```

36. Write a C/C++ function to print of a linear linked list in reverse order and also count the number of nodes...

```
// Function returns count of nodes
int printReverseAndCount(struct Node* head) {
    if (head == NULL)
        return 0; // Base case

    // Recursive call + 1 for counting
    int count = 1 + printReverseAndCount(head->next);

    // Print after recursive call returns (Reverse Order)
    printf("%d ", head->data);

    return count;
}
```

37. Write a C/C++ code for insertion of a data item after a specified data item into a doubly linked list.

```
void insertAfterDoubly(struct Node* head, int key, int newData) {
    struct Node* temp = head;
    while (temp != NULL && temp->data != key)
        temp = temp->next;

    if (temp == NULL) return; // Key not found

    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = newData;

    newNode->next = temp->next;
    newNode->prev = temp;

    if (temp->next != NULL) { // If not inserting at the very end
        temp->next->prev = newNode;
    }
    temp->next = newNode;
}
```

38. Write an algorithm to delete any node from the list, assuming the key value of the node to be deleted is known...

Algorithm: Delete_Node_By_Key(Head, Key)

1. **Validate:** IF Head is NULL, Print "List Empty" and Exit.
2. **Handle Head:** IF Head->data == Key:
    - Set Temp = Head.
    - Set Head = Head->next.
    - Free Temp.
    - Exit.
3. **Traverse:** Initialize Prev = Head and Curr = Head->next.
4. **Loop:** WHILE Curr is NOT NULL AND Curr->data != Key:
    - Move Prev to Curr.
    - Move Curr to Curr->next.

5. **Check Found:** IF Curr is NULL, Print "Key Not Found" and Exit.
6. **Unlink:** Set Prev->next = Curr->next.
7. **Free:** Free memory of Curr.

39. Write a function to delete a node from a doubly linked list.

```
void deleteNode(struct Node** head_ref, struct Node* del) {
    if (*head_ref == NULL || del == NULL) return;

    // If node to be deleted is head node
    if (*head_ref == del)
        *head_ref = del->next;

    // Change next only if node to be deleted is NOT the last node
    if (del->next != NULL)
        del->next->prev = del->prev;

    // Change prev only if node to be deleted is NOT the first node
    if (del->prev != NULL)
        del->prev->next = del->next;

    free(del);
}
```

40. Write a function to insert a node at the end of the singly linked list.

```
void insertAtEnd(struct Node** head_ref, int newData) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    struct Node* last = *head_ref;
    newNode->data = newData;
    newNode->next = NULL;

    if (*head_ref == NULL) { // If list is empty
        *head_ref = newNode;
        return;
    }

    while (last->next != NULL) // Traverse to tail
        last = last->next;

    last->next = newNode;
}
```

41. Write a function to delete last node of the singly linked list.

```
void deleteLast(struct Node** head_ref) {
    if (*head_ref == NULL) return; // Empty list

    if ((*head_ref)->next == NULL) { // Only one node
        free(*head_ref);
        *head_ref = NULL;
        return;
    }

    struct Node* second_last = *head_ref;
```

```
    while (second_last->next->next != NULL) // Stop at 2nd last node
        second_last = second_last->next;

    free(second_last->next); // Free last node
    second_last->next = NULL; // Set 2nd last as tail
}
```

42. Write a function to insert a node at the beginning of the circular linked list.

```
void insertAtBeginCircular(struct Node** head_ref, int data) {
    struct Node* ptr1 = (struct Node*)malloc(sizeof(struct Node));
    ptr1->data = data;
    struct Node* temp = *head_ref;

    if (*head_ref == NULL) {
        ptr1->next = ptr1; // Point to itself
        *head_ref = ptr1;
    } else {
        // Find last node to update its next pointer
        while (temp->next != *head_ref)
            temp = temp->next;

        ptr1->next = *head_ref;
        temp->next = ptr1;
        *head_ref = ptr1; // Update head
    }
}
```

43. Write a function to insert data in a sorted linked list.

```
void sortedInsert(struct Node** head_ref, struct Node* newNode) {
    struct Node* current;
    // Case 1: Insert at head
    if (*head_ref == NULL || (*head_ref)->data >= newNode->data) {
        newNode->next = *head_ref;
        *head_ref = newNode;
    }
    else {
        // Case 2: Locate position
        current = *head_ref;
        while (current->next != NULL && current->next->data < newNode-
>data) {
            current = current->next;
        }
        newNode->next = current->next;
        current->next = newNode;
    }
}
```

44. The following C function takes a singly linked list of integers as a parameter and rearranges the elements of the list... What will be the contents of the list after the function completes execution?

Answer: 2, 1, 4, 3, 6, 5, 7

- **Explanation:** The function iterates through the list and swaps the **values** (not the nodes themselves) of every adjacent pair (p and q).
    - Pair (1, 2) becomes (2, 1).
    - Pair (3, 4) becomes (4, 3).
    - Pair (5, 6) becomes (6, 5).
    - Node 7 remains unchanged because q becomes NULL (no pair).

45. Supposed the linked list in the memory consisting of numerical values. Write a function for each of the following:

(i) To find the maximum (MAX) of the values in the list.

(ii) To find the average (MEAN) of the values in the list.

(iii) To find the product (PROD) of the values in the list.

***Answer*:**

```
// (i) Function to find Maximum
int findMax(struct node *head) {
    if (head == NULL) return -1; // Error code or handle empty list
    int max = head->value;
    struct node *current = head->next;
    while (current != NULL) {
        if (current->value > max)
            max = current->value;
        current = current->next;
    }
    return max;
}

// (ii) Function to find Mean
float findMean(struct node *head) {
    if (head == NULL) return 0.0;
    int sum = 0, count = 0;
    struct node *current = head;
    while (current != NULL) {
        sum += current->value;
        count++;
        current = current->next;
    }
    return (float)sum / count;
}

// (iii) Function to find Product
long long findProduct(struct node *head) {
    if (head == NULL) return 0;
    long long prod = 1;
    struct node *current = head;
    while (current != NULL) {
        prod *= current->value;
        current = current->next;
    }
    return prod;
}
```

46. How to detect a cycle in a linked list?
Method: Floyd's Cycle-Finding Algorithm (Tortoise and Hare).
Algorithm:

1. Initialize two pointers, Slow and Fast, both pointing to the Head of the list.
2. Traverse the list:
    - Move Slow pointer **one** step (Slow = Slow->next).
    - Move Fast pointer **two** steps (Fast = Fast->next->next).
3. **Check:**
    - If Fast reaches NULL, there is **No Cycle**.
    - If Fast becomes equal to Slow (Fast == Slow), a **Cycle Detected** exists.

47. Given a singly linked list A, find the middle element of the linked list.
Algorithm:

1. Initialize two pointers, slow_ptr and fast_ptr, to the head of the list.
2. Move fast_ptr by two nodes and slow_ptr by one node.
3. Repeat step 2 until fast_ptr reaches the end (NULL) or the last node (fast_ptr->next == NULL).
4. At this point, slow_ptr will be pointing to the middle element.
    - *Note:* If the list has an even number of nodes, slow_ptr will point to the second middle element (depending on implementation).

48. Explain insertion and deletion operations on linked stack and linked queue. Write an algorithm to add two polynomials using dynamic memory allocation. Polynomials can be represented either by an array or by linked list. Compare and contrast these two types of representations. Represent the following polynomials by a linked data structure (show only the diagram).

Part A: Operations on Linked Stack and Queue

- **Linked Stack:**
    - **Insertion (Push):** Create a new node. Set newNode->next = Top. Update Top = newNode. (Happens at the Head).
    - **Deletion (Pop):** Check if empty. If not, temp = Top. Update Top = Top->next. Free temp. (Happens at the Head).
- **Linked Queue:**
    - **Insertion (Enqueue):** Create a new node. If Queue is empty, Front = Rear = newNode. Else, Rear->next = newNode, then Rear = newNode. (Happens at the Rear).
    - **Deletion (Dequeue):** Check if empty. If not, temp = Front. Update Front = Front->next. Free temp. (Happens at the Front).

Part B: Algorithm to Add Two Polynomials
Assumptions: Polynomial nodes have fields coeff, exp, and next. Lists are sorted by exponent descending.

1. Initialize ptr1 = Poly1, ptr2 = Poly2, and ResultHead = NULL.
2. **While** (ptr1 != NULL AND ptr2 != NULL):
    - **If** ptr1->exp == ptr2->exp:

        - Sum = ptr1->coeff + ptr2->coeff.
        - If Sum != 0, Insert node (Sum, ptr1->exp) into ResultHead.
        - Move ptr1 and ptr2 forward.
    - **If** ptr1->exp > ptr2->exp:
        - Insert node (ptr1->coeff, ptr1->exp) into ResultHead.
        - Move ptr1 forward.
    - **If** ptr1->exp < ptr2->exp:
        - Insert node (ptr2->coeff, ptr2->exp) into ResultHead.
        - Move ptr2 forward.
3. **While** ptr1 != NULL: Insert remaining nodes from Poly1 to Result.
4. **While** ptr2 != NULL: Insert remaining nodes from Poly2 to Result.

Part C: Comparison (Array vs. Linked List for Polynomials)

Feature	Array Representation	Linked List Representation
Space Efficiency	**Wasteful for sparse polynomials** (e.g., $x^{100} + 1$ requires array size 101).	**Efficient**, stores only non-zero terms (only 2 nodes for $x^{100}+ 1$).
Size	Fixed max degree size.	Dynamic growth.
Complexity	Easier to implement.	Complex pointer manipulation.
Access	Random access (O(1)) to any exponent.	Sequential access (O(N)).

49. Dene a linked list with a loop as a linked list in which the tail element points to one of the list's element and not to NULL. Assume that you are given a linked list L, and two pointers P1, P2 to the head. Write an algorithm that decides whether the list has a loop without modifying the original list. The algorithm should run in time O (n)., where n is the number of elements in the list.

Algorithm: Detect_Loop(Head)

1. **Initialize** two pointers P1 (Slow) and P2 (Fast).
2. Set P1 = Head and P2 = Head.
3. **Loop While** P1, P2, and P2->next are not NULL:
    - Advance P1 by 1 step: P1 = P1->next.
    - Advance P2 by 2 steps: P2 = P2->next->next.
    - **Compare:** IF P1 == P2:
        - Return **TRUE** (Loop detected).
4. **End Loop.**
5. Return **FALSE** (Pointer reached NULL, so no loop).

50. In the worst case, the number of comparisons needed to search a singly linked list of length n for a given element is which of the following ?
Then explain.
(A) log 2 n
(B) n/2

(C) log 2 n 1
(D) nCorrect

Answer Option: (D) n
Explanation:
A Singly Linked List does not support random access (like an array). To find an element, you must traverse sequentially from the Head node.

- **Best Case:** The element is at the Head (1 comparison).
- **Worst Case:** The element is at the very end, or not in the list at all. You must visit every node. Therefore, the complexity is Linear O(n).

51. How does a stack implemented using a linked list differ from a stack implemented using an array?

Feature	**Stack (Array Implementation)**	**Stack (Linked List Implementation)**
Size Limit	**Fixed Size.** Can result in Stack Overflow if MAX is reached.	**Dynamic Size.** Grows until system heap memory is exhausted.
Memory Usage	Efficient for data, but may waste space if array is large and stack is empty.	Requires extra memory for a **pointer** in every node.
Performance	Slightly faster due to cache locality and no dynamic allocation overhead.	Slightly slower due to malloc/free calls and pointer dereferencing.
Operations	Push/Pop check for array bounds (Top == MAX-1).	Push/Pop check for Heap memory availability (malloc == NULL).

Searching & Sorting

1. Merge sort uses which of the following technique to implement sorting?
a) backtracking
b) greedy algorithm
c) divide and conquer
d) dynamic programming

2. What is the average case time complexity of merge sort?
a) $O(nlogn)$ b) $O(n^2)$ c) $O(n^2logn)$ d)$O(nlogn^2)$

3. What is the auxiliary space complexity of merge sort?
a) $O(1)$ b) $O(logn)$ c) $O(n)$ d) $O(nlogn)$

4. What is the worst case time complexity of merge sort?
a) $O(nlogn)$ b) $O(n^2)$ c) $O(n^2logn)$ d)$O(nlogn^2)$

5. Which of the following method is used for sorting in merge sort?
a) merging b) partitioning c) selection d) exchanging

6. What will be the best case time complexity of merge sort?
a) $O(nlogn)$ b) $O(n^2)$ c) $O(n^2logn)$ d)$O(nlogn^2)$

7. Choose the incorrect statement about merge sort from the following?
a) it is a comparison based sort b) it is an adaptive algorithm
c) it is not an in place algorithm d) it is stable algorithm

8. Which of the following is not in-place sorting algorithm by default?
a) merge sort b) quick sort c) heap sort d) insertion sort

9. Which of the following is not a stable sorting algorithm?
a) Quick sort b) Cocktail sort c) Bubble sort d) Merge sort

10. Which of the following stable sorting algorithm takes the least time when applied to an almost sorted array?
a) Quick sort b) Insertion sort c) Selection sort d) Merge sort

11. Choose the correct code for merge sort.

a)
```
void merge_sort(int arr[], int left, int right){
      if (left > right){
            int mid = (right-left)/2;
            merge_sort(arr, left, mid);
            merge_sort(arr, mid+1, right);
            merge(arr, left, mid, right);
      }
}
```

b)
```
void merge_sort(int arr[], int left, int right){
      if (left < right){
            int mid = left+(right-left)/2;
            merge_sort(arr, left, mid);
            merge_sort(arr, mid+1, right);
            merge(arr, left, mid, right);
      }
}
```

c)
```
void merge_sort(int arr[], int left, int right){
      if (left < right){
            int mid = left+(right-left)/2;
            merge(arr, left, mid, right);
            merge_sort(arr, left, mid);
            merge_sort(arr, mid+1, right);
      }
}
```

d)
```
void merge_sort(int arr[], int left, int right){
      if (left < right){
            int mid = (right-left)/2;
            merge(arr, left, mid, right);
            merge_sort(arr, left, mid);
            merge_sort(arr, mid+1, right);
      }
}
```

Correct Answer: (a)

12. Which of the following sorting algorithms is the fastest?

a) Merge sort b) Quick sort c) Insertion sort d) Shell sort

13. Quick sort follows Divide-and-Conquer strategy.

a) True b) False

14. What is the worst case time complexity of a quick sort algorithm?

a) $O(N)$ b) $O(N \log N)$ c) $O(N^2)$ d) $O(\log N)$

15. Which of the following methods is the most effective for picking the pivot element?

a) first element b) last element

c) median-of-three partitioning d) random element

16. Find the pivot element from the given input using median-of-three partitioning method.

8, 1, 4, 9, 6, 3, 5, 2, 7, 0.

a) 8 b) 7 c) 9 d) 6

17. Which is the safest method to choose a pivot element?

a) choosing a random element as pivot b) choosing the first element as pivot

c) choosing the last element as pivot d) median-of-three partitioning method

18. What is the average running time of a quick sort algorithm?

a) $O(N^2)$ b) $O(N)$ c) $O(NlogN)$ d) $O(logN)$

19. Which of the following sorting algorithms is used along with quick sort to sort the sub arrays?

a) Merge sort b) Shell sort c) Insertion sort d) Bubble sort

20. Quick sort uses join operation rather than merge operation.

a) true b) false

21. How many sub arrays does the quick sort algorithm divide the entire array into?

a) one b) two c) three d) four

22. Which is the worst method of choosing a pivot element?

a) first element as pivot b) last element as pivot

c) median-of-three partitioning d) random element as pivot

23. Which among the following is the best cut-o range to perform insertion sort within a quick sort?

a) N = 0 – 5 b) N = 5 – 20 c) N = 20 – 30 d) N > 30

24. Quick sort is a

a) greedy algorithm b) divide and conquer algorithm

c) dynamic programming algorithm d) backtracking algorithm

25. What is the worst case time complexity of the Quick sort?

a) $O(nlogn)$ b) $O(n)$ c) $O(n^3)$ d)$O(n^2)$

26. Apply Quick sort on a given sequence 7 11 14 6 9 4 3 12. What is the sequence after first phase, pivot is first element?

a) 6 4 3 7 11 9 14 12 b) 6 3 4 7 9 14 11 12

c) 7 6 14 11 9 4 3 12 d) 7 6 4 3 9 14 11 12

27. The best case behaviour occurs for quick sort is, if partition splits the array of size n into

a) n/2 : (n/2) – 1 b) n/2 : n/3 c) n/4 : 3n/2 d) n/4 : 3n/4

28. Quick sort is a stable sorting algorithm.

a) True b) False

29. Consider the Quick sort algorithm in which the partitioning procedure splits elements into two sub-arrays and each sub-array contains at least one-fourth of the elements. Let T(n) be the number of comparisons required to sort array of n elements. Then T (n) <=?

a) $T(n) \leq 2T(n/4) + cn$ b) $T(n) \leq T(n/4) + T(3n/4) + cn$

c) $T(n) \leq 2T(3n/4) + cn$ d) $T(n) \leq T(n/3) + T(3n/4) + cn$

30. Consider the Quick sort algorithm which sorts elements in ascending order using the first element as pivot. Then which of the following input sequence will require a maximum number of comparisons when this algorithm is applied on it?

a) 22 25 56 67 89 b) 52 25 76 67 89

c) 22 25 76 67 50 d) 52 25 89 67 76

31. A machine needs a minimum of 200 sec to sort 1000 elements by Quick sort. The minimum time needed to sort 200 elements will be approximately -

a) 60.2 sec b) 45.54 sec c) 31.11 sec d) 20 sec

32. Which one of the following sorting algorithm is best suited to sort an array of 1 million elements?

a) Bubble sort b) Insertion sort c) Merge sort d) Quick sort

33. Quick sort is a space-optimised version of -

a) Bubble sort b) Selection sort c) Insertion sort d) Binary tree sort

34. Quick Sort can be categorized into which of the following?

a) Brute Force technique b) Divide and conquer

c) Greedy algorithm d) Dynamic programming

35. What is the worst case complexity of Quick Sort?

a) $O(n\log n)$ b) $O(\log n)$ c) $O(n)$ d) $O(n^2)$

36. What is a randomized Quick Sort?

a) The leftmost element is chosen as the pivot

b) The rightmost element is chosen as the pivot
c) Any element in the array is chosen as the pivot
d) A random number is generated which is used as the pivot

37. What is the best case complexity of Quick Sort?
a) O(nlogn) b) O(logn) c) O(n) d) $O(n^2)$

38. The given array is arr = {2, 3, 4, 1, 6}. What are the pivots that are returned as a result of subsequent partitioning?
a) 1 and 3 b) 3 and 1 c) 2 and 6 d) 6 and 2

39. What is the average case complexity of Quick Sort?
a) O(nlogn) b) O(logn) c) O(n) d) $O(n^2)$

40. The given array is arr = {2, 6, 1}. What are the pivots that are returned as a result of subsequent partitioning?
a) 1 and 6 b) 6 and 1 c) 2 and 6 d) 1

41. Which of the following is not true about Quick Sort?
a) in-place algorithm b) pivot position can be changed
c) adaptive sorting algorithm d) can be implemented as a stable sort

1. What is the difference between Linear & Binary Search? What is the prerequisite for binary search?

- **Differences:**
  - **Mechanism:** Linear search scans elements sequentially from the start to the end. Binary search uses a divide-and-conquer approach, repeatedly dividing the search interval in half.
  - **Time Complexity:** Linear search has a worst-case time complexity of $O(n)$, whereas Binary search is significantly faster with O(log n).
  - **Data Access:** Linear search can be performed on both random-access (arrays) and sequential-access (linked lists) data structures. Binary search requires random access to be efficient.
- **Prerequisite for Binary Search:**
  - The array or list **must be sorted** (either in ascending or descending order).
  - Direct access to the middle element (random access) is required for optimal performance.

2. What do you mean by complexity? What is time complexity and what is space complexity?

- **Complexity:** In computer science, complexity refers to the amount of computational resources (time or memory) required by an algorithm to run to completion, usually expressed as a function of the input size (n).

- **Time Complexity:** This quantifies the amount of time (or number of primitive operations) an algorithm takes to run as a function of the input length. It estimates the growth rate of execution time (e.g., $O(n)$, $O(n^2)$).
- **Space Complexity:** This quantifies the amount of working memory (RAM) an algorithm needs to execute, including inputs and auxiliary memory, as a function of the input size.

3. Explain the meaning of worst-case analysis and best-case analysis with an example.

- **Worst-Case Analysis (Big O):** Represents the scenario where the algorithm takes the maximum amount of time or steps to complete. It defines the upper bound of the algorithm's performance.
    - *Example:* In Linear Search, the worst case occurs when the target element is at the very end of the list or not present at all, requiring n comparisons.
- **Best-Case Analysis (Big Omega):** Represents the scenario where the algorithm completes in the minimum number of steps.
    - *Example:* In Linear Search, the best case occurs when the target element is found at the very first index (Index 0), requiring only 1 comparison.

4. What are the advantages of binary search over linear search?

- **Efficiency on Large Datasets:** Binary search reduces the search space by half in every step. For 1,000,000 elements, Linear search might make 1,000,000 comparisons, while Binary search will make at most ≈ 20.
- **Logarithmic Growth:** The time complexity is $O(\log n)$, making it scalable for massive databases where $O(n)$ would be too slow.
- **Predictable Performance:** While the best case for linear search is $O(1)$, its average performance is $O(n/2)$. Binary search maintains high efficiency even in average scenarios.

5. What is the precondition of performing binary search in an array? Write the binary search algorithm.

- **Precondition:** The input array must be strictly sorted.
- **Algorithm Steps:**
    1. Initialize two pointers: low at the first index (0) and high at the last index (n-1).
    2. Calculate the middle index: mid = low + (high - low) / 2.
    3. **Compare:** Check if array[mid] equals the target value.
        - If **True**: Return mid (Search successful).
        - If **False**: Check if the target is smaller than array[mid].
            - If smaller: Update high = mid - 1 (Ignore right half).
            - If larger: Update low = mid + 1 (Ignore left half).
    4. Repeat steps 2-3 while low <= high.
    5. If the loop ends without finding the target, return -1 (Element not found).

6. Explain why "Binary search technique can't be implemented using linked list." Justify your answer.

- **Random Access Requirement:** Binary search relies on calculating a middle index (k) and jumping directly to that memory location in $O(1)$ time. Arrays support this via pointer arithmetic.
- **Sequential Access Limitation:** In a Linked List, to reach the Kth element (the middle), you must traverse from the head node, taking $O(k)$ time.
- **Efficiency Negation:** Even if the list is sorted, finding the middle element takes linear time. Consequently, the recurrence relation becomes $T(n) = T(n/2) + O(n/2)$, which solves to $O(n)$. This negates the $O(\log n)$ advantage, making it no better than linear search.

7. Derive the worst-case time complexity of binary search and explain each step.

- **The Logic:** In the worst case, the target element is not found, or is found at the last possible split. The problem size n is reduced by half in every iteration.
- **Derivation:**
    1. Iteration 1: Length = n
    2. Iteration 2: Length = $n/2$
    3. Iteration 3: Length = $n/2^2$
    4. ...
    5. Iteration k: Length = $n/2^k$
- Stopping Condition: The search stops when the array size becomes 1. Therefore:

$$n/2^k = 1$$
$$n = 2^k$$

- Solving for k: Taking the logarithm (base 2) on both sides:

$$\text{Log}_2(n) = \log_2(2^k)$$
$$k = \log_2(n)$$

- **Result:** Since k represents the number of steps, the worst-case time complexity is **$O(\log n)$**.

8. Discuss how binary search is different from linear search in terms of its working mechanism and efficiency.

- **Working Mechanism:**
    - **Linear Search:** Works on an iterative checking mechanism. It is exhaustive, checking every single element one by one until a match is found or the list ends.
    - **Binary Search:** Works on a "Divide and Conquer" mechanism. It compares the target with the middle element and discards half of the array immediately, narrowing the search scope drastically.
- **Efficiency:**
    - **Linear Search:** Efficiency drops linearly as data grows ($O(n)$). If data doubles, time doubles.
    - **Binary Search:** Efficiency is logarithmic ($O(\log n)$). If data doubles, time only increases by **one** constant unit/step.

9. Given a sorted array of integers, write a program to search for a specific element using binary search.

```
#include <stdio.h>

int binarySearch(int arr[], int size, int target) {
    int low = 0, high = size - 1;

    while (low <= high) {
        // Prevent overflow for large integers
        int mid = low + (high - low) / 2;

        if (arr[mid] == target)
            return mid; // Return index

        if (arr[mid] < target)
            low = mid + 1; // Search right half
        else
            high = mid - 1; // Search left half
    }
    return -1; // Not found
}

int main() {
    int arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 23;
    int result = binarySearch(arr, n, target);

    if (result != -1)
        printf("Element found at index %d", result);
    else
        printf("Element not found");

    return 0;
}
```

10. Implement an iterative and recursive version of binary search and compare their performance.

- **Iterative Version:** (As shown in the previous answer) uses a while loop.
  - *Space Complexity:* O(1) (Auxiliary space).
  - *Performance:* Generally preferred in C due to no function call overhead.
- **Recursive Version:**

```
int recursiveBinarySearch(int arr[], int low, int high, int target) {
    if (high >= low) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] > target)
            return recursiveBinarySearch(arr, low, mid - 1, target);
        return recursiveBinarySearch(arr, mid + 1, high, target);
    }
    return -1;
```

}

- *Space Complexity:* O(log n) due to stack memory used for recursive function calls.
- *Performance:* Slightly slower due to the overhead of pushing/popping stack frames, and risks stack overflow on extremely large datasets.

11. Analyse how binary search would perform if applied to a nearly sorted array. Provide code and results for the analysis.

- **Analysis:** Standard binary search is **unreliable** and incorrect for nearly sorted arrays. The algorithm makes a definitive decision to go Left or Right based on mid. If an element is slightly out of place (e.g., swapped with a neighbour), the algorithm might discard the half containing the target.
- **Result:** It may fail to find an element that actually exists in the array.
- **Example Logic:**
  - Array: [10, 30, 20, 40, 50] (Sorted except 20 and 30).
  - Target: 20.
  - Mid is 20. Found? Yes.
  - *However*, if we search for 30: Mid is 20. 30 > 20, so we look right (40, 50). We never look left where 30 is actually located.
- **Conclusion:** Standard Binary Search cannot be applied; a modified approach (checking neighbours mid-1, mid+1) is required for nearly sorted data.

12. Given a dataset of 1000 sorted numbers, determine the number of comparisons binary search will make in the worst-case scenario.

- **Formula:** The worst-case number of comparisons in binary search is given by $[\log_2 n] + 1$.
- **Calculation:**
  - n = 1000
  - $\log_2(1000) \approx 9.966$
  - Since $2^9 = 512$ and $2^{10} = 1024$.
  - To cover 1000 elements, we need the power of 10.
- **Result:** It will take a maximum of **10 comparisons**.

13. Suppose you are designing a search feature for a large database. Discuss the scenarios where binary search is more suitable than linear search and vice versa, with examples.

- **Use Binary Search When:**
  - **Data is Static:** The database is updated infrequently (e.g., historical records, dictionaries). The cost of sorting once is justified by fast repeated searches.
  - **Volume is High:** Searching millions of records (e.g., finding a user ID in a sorted user table). The speed difference between O(n) and O(log n) is critical.
  - **Memory is Available:** Standard arrays are used, or data is indexed.
- **Use Linear Search When:**

- **Data is Dynamic:** The database receives constant real-time insertions/deletions (e.g., live logs). Maintaining a sorted order for binary search would be too expensive ($O(n)$ per insertion).
- **Dataset is Small:** If $n < 50$, linear search is often faster due to CPU caching and lack of overhead.
- **Data is Unsorted/Complex:** When searching by a non-indexed key (e.g., searching a sorted ID list by "Last Name").

14. Explain how binary search can be adapted for searching in a rotated sorted array. Write a pseudo-code for this adapted binary search and explain the logic.

- **The Logic:** In a rotated sorted array (e.g., [4, 5, 6, 7, 0, 1, 2]), at least one half (left or right) of the array is always sorted. We determine which half is sorted, check if the target lies in that range, and adjust pointers accordingly.
- **Pseudo-code:**

```
Function RotatedSearch(arr, target):
    low = 0, high = n-1
    While low <= high:
        mid = (low + high) / 2
        If arr[mid] == target: Return mid

        // Check if Left half is sorted
        If arr[low] <= arr[mid]:
            If target >= arr[low] AND target < arr[mid]:
                high = mid - 1  // Target is in the left sorted half
            Else:
                low = mid + 1   // Target is in the right half

        // Otherwise, Right half must be sorted
        Else:
            If target > arr[mid] AND target <= arr[high]:
                low = mid + 1   // Target is in the right sorted half
            Else:
                high = mid - 1  // Target is in the left half
    Return -1
```

15. Define sorting. What is the importance of sorting?

- **Definition:** Sorting is the process of arranging data items in a specific order, typically ascending or descending, based on a defined key (numerical, alphabetical, etc.).
- **Importance:**
    - **Efficient Searching:** Sorting allows the use of optimized search algorithms like Binary Search ($O(\log n)$) instead of Linear Search ($O(n)$).
    - **Data Analysis:** It simplifies tasks like finding duplicates, finding the maximum/minimum, and calculating frequency distributions.
    - **Problem Solving:** Many complex algorithms (like Kruskal's algorithm for Minimum Spanning Trees) rely on sorted input data.

2. What are the different types of sorting techniques? Which sorting technique has the least worst case?

- **Types of Sorting:**
  - **Internal Sorting:** Data fits entirely in main memory (e.g., Bubble, Quick, Insertion).
  - **External Sorting:** Data is too large for RAM and resides on secondary storage (e.g., Merge Sort variants).
- **Least Worst Case:**
  - **Merge Sort** and **Heap Sort** have the best guaranteed worst-case time complexity of **O(nlog n)**. Unlike Quick Sort, their performance does not degrade to $O(n^2)$ regardless of input order.

3. Compare heap sort and quick sort.

Feature	Heap Sort	Quick Sort
Worst Case	O(nlog n)	$O(n^2)$
Average Case	O(nlog n)	O(nlog n)
Space Complexity	O(1) (In-place)	O(log n) (Stack)
Stability	Unstable	Unstable
Speed	Generally slower due to cache locality issues.	Usually faster in practice due to good locality of reference.

4. Sort the elements 77, 49, 25, 12, 9, 33, 56, 81 using various sorting techniques.

(Note: In an exam, you would perform the step-by-step trace for a specific requested algorithm. Below is the final sorted result which applies to all).

- **Input Array:** [77, 49, 25, 12, 9, 33, 56, 81]
- **Sorted Output:** [9, 12, 25, 33, 49, 56, 77, 81]

5. Sort the following sequence of numbers in descending order using heap sort: 42, 34, 75, 23, 21, 18, 90, 67, 78.

(To sort descending, we typically use a Min-Heap. The root (minimum) is swapped with the last element, and the size is reduced).

1. **Build Min Heap:** [18, 21, 42, 23, 34, 75, 90, 67, 78] (Structure may vary slightly based on logic).
2. **Extraction Phase:**
   - Swap 18 (Min) with last element. Heapify remaining.
   - Repeat until heap is empty.
3. **Final Descending Order:** 90, 78, 75, 67, 42, 34, 23, 21, 18

6. Identify the sorting algorithm applied to a given rearrangement of a data set after two passes.

- **Bubble Sort:** The largest 2 elements will be sorted at the end of the array.

- **Selection Sort:** The smallest 2 elements will be sorted at the beginning of the array.
- **Insertion Sort:** The first 3 elements (Indices 0, 1, 2) will be sorted relative to each other.

1. Explain the difference between bubble sort and quick sort. Which one is more efficient?

- **Differences:**
  - **Mechanism:** Bubble sort compares adjacent elements and swaps them if they are in the wrong order (iterative). Quick sort picks a "pivot" element and partitions the array into elements smaller and larger than the pivot (recursive/divide and conquer).
  - **Swapping:** Bubble sort performs excessive swapping. Quick sort performs fewer swaps, moving elements over large distances.
- **Efficiency:**
  - **Quick Sort is significantly more efficient.**
  - Bubble Sort is $O(n^2)$, making it impractical for large datasets. Quick Sort is $O(n\log n)$ on average and is the industry standard for general-purpose sorting.

3. Quick sort shows quadratic behaviour in certain situations. Justify.

- **Justification:** Quick Sort relies on the pivot splitting the array into roughly equal halves.
- **Worst Case Scenario ($O(n^2)$):** Occurs when the pivot chosen is always the smallest or largest element in the list (highly unbalanced partition).
  - **Example:** If the input is **already sorted** (ascending or descending) and the algorithm always picks the **first element** as the pivot, the recursion tree becomes a skewed chain (depth n) rather than a balanced tree (depth log n).

4. If the following sequence of numbers is to be sorted using quick sort, show the iterations: 42, 34, 75, 23, 21, 18, 90, 67, 78.

(Assuming First Element as Pivot)

- **Pass 1 (Pivot = 42):**
  - Move elements < 42 to left, > 42 to right.
  - Result: [18, 34, 21, 23] 42 [75, 90, 67, 78]
- **Pass 2 (Recursively sort Left [18, 34, 21, 23]):**
  - Pivot = 18. Result: 18 [34, 21, 23]
- **Pass 3 (Recursively sort Right [75, 90, 67, 78]):**
  - Pivot = 75. Result: [67] 75 [90, 78]
- *(Process continues recursively until sub-arrays are size 0 or 1).*
- **Final:** 18, 21, 23, 34, 42, 67, 75, 78, 90

5. In which situation insertion sort running time complexity $O(n)$? Explain with example. Then the same situation for quick sort, what happens?

- **Insertion Sort O(n):** This happens in the **Best Case**, which is when the array is **already sorted**. The inner while loop condition fails immediately for every element, resulting in only n-1 comparisons and 0 swaps.
- **Quick Sort in the same situation:** If the array is already sorted and the **first element** is chosen as the pivot, Quick Sort hits its **Worst Case**.
    - The pivot will be the minimum element.
    - The partition will produce one empty sub-array and one sub-array of size n-1.
    - Time complexity degrades to $O(n^2)$.

6. Why merge sort is called a divide and conquer method?

- Merge sort strictly follows the three steps of the Divide and Conquer paradigm:
    1. **Divide:** The problem (array) is divided into two equal halves (sub-problems) by calculating the midpoint.
    2. **Conquer:** The sub-problems are solved recursively (sorting the two halves).
    3. **Combine:** The sorted halves are merged back together to form the final sorted solution.

1. Sort the following sequence of numbers using merge sort: 66, 77, 11, 88, 99, 22, 33, 44, 55.

- **Divide Phase:**
    - [66, 77, 11, 88, 99] | [22, 33, 44, 55]
    - [66, 77] [11, 88, 99] | [22, 33] [44, 55]
- **Conquer & Merge Phase:**
    - [66, 77] merged with [11, 88, 99] -> [11, 66, 77, 88, 99]
    - [22, 33] merged with [44, 55] -> [22, 33, 44, 55]
- **Final Merge:**
    - Compare heads of both lists (11 vs 22, 66 vs 22, etc.).
    - Result: 11, 22, 33, 44, 55, 66, 77, 88, 99

2. Write a recursive function to perform selection sort.

```
void recursiveSelectionSort(int arr[], int n, int index) {
    if (index == n) return; // Base Case

    int minIndex = index;
    // Find min in the unsorted subarray
    for (int j = index + 1; j < n; j++) {
        if (arr[j] < arr[minIndex])
            minIndex = j;
    }
    // Swap
    int temp = arr[index];
    arr[index] = arr[minIndex];
    arr[minIndex] = temp;

    // Recur for the next index
    recursiveSelectionSort(arr, n, index + 1);
```

```
}
```

3. Write a C function for selection sort and also calculate the time complexity.

```
void selectionSort(int arr[], int n) {
    int i, j, min_idx;
    for (i = 0; i < n-1; i++) {
        min_idx = i;
        for (j = i+1; j < n; j++)
            if (arr[j] < arr[min_idx])
                min_idx = j;

        // Swap
        int temp = arr[min_idx];
        arr[min_idx] = arr[i];
        arr[i] = temp;
    }
}
```

- **Complexity Calculation:**
  - Outer loop runs n-1 times.
  - Inner loop runs n-1, n-2, ……, 1 times.
  - Sum of natural numbers: $S = (n-1) + (n-2) + \ldots\ldots + 1 = n(n-1) / 2 \approx n^2/2$.
  - **Time Complexity:** $O(n^2)$.

4. Write the algorithm to sort an array of integers using the Insertion Sort method. Explain time complexity. Explain principle operation of Quick sort.

- **Insertion Sort Algorithm:**
  1. Iterate from arr[1] to arr[n].
  2. Compare the current element (key) to its predecessor.
  3. If the key is smaller than the predecessor, compare it to the elements before. Move greater elements one position up to make space.
  4. Insert key at the correct position.
- **Time Complexity:**
  - Worst/Average: $O(n^2)$ (Reverse sorted or random).
  - Best: $O(n)$ (Already sorted).
- **Quick Sort Principle:**
  - It relies on **Partitioning**. A pivot is chosen, and the array is rearranged such that all smaller elements are on the left and all larger elements are on the right. This places the pivot in its correct sorted position. The process is repeated for the left and right sub-arrays.

5. Write short notes on Radix Sort.

- **Non-Comparative:** Radix sort does not compare elements (no a > b logic). It works by distributing elements into buckets based on their digits.
- **Mechanism:** It processes numbers digit by digit, usually starting from the Least Significant Digit (LSD) to the Most Significant Digit (MSD).
- **Complexity:** $O(nk)$ where n is the number of elements and k
- is the number of digits in the largest number. It can be faster than Quick Sort for integers.

6. What is a heap? Define max and min heap. Explain with an example how to construct a heap.

- **Heap:** A complete binary tree that satisfies the heap property.
- **Max Heap:** The root node is greater than or equal to its children (recursively true for all nodes). Root = Max.
- **Min Heap:** The root node is less than or equal to its children. Root = Min.
- **Construction (Max Heap Example):**
  - Input: [10, 20, 15]
  - Insert 10.
  - Insert 20 (Left child). 20 > 10, so Swap. Root is 20.
  - Insert 15 (Right child). 15 < 20. No Swap.
  - Result: Root 20, Children 10 and 15.

7. Define Big-O, Ω, Θ notation. Prove that O(f(x)) + O(g(x)) = O(max(f(x), g(x))).

- **Big-O (O):** Upper bound. Worst-case complexity.
- **Omega (Ω):** Lower bound. Best-case complexity.
- **Theta (Θ):** Tight bound. Average/Exact case behaviour.
- **Proof:**
  - Let $T_1(n) \in O(f(n))$ and $T_2(n) \in O(g(n))$.
  - This implies $T_1(n) \leq c_1.f(n)$ and $T_2(n) \leq c_2.g(n)$ for $n \geq n_0$.
  - $T_1(n) + T_2(n) \leq c_1.f(n) + c_2.g(n)$
  - Let $h(n) = max(f(n), g(n))$. Then $f(n) \leq h(n)$ and $g(n) \leq h(n)$.
  - $T_1(n) + T_2(n) \leq (c_1 + c_2).h(n)$ = $C\ .\ max(f(n), g(n))$.
  - Therefore, sum is $O(max(f(n), g(n)))$.

1. Compare the running time complexity of different sorting algorithms.

Algorithm	Best Case	Average Case	Worst Case
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

2. Discuss the advantages of insertion sort.

- **Adaptive:** It is efficient ($O(n)$) for data sets that are already substantially sorted.
- **Online:** It can sort a list as it receives it, element by element.
- **Low Overhead:** For very small datasets ($n \leq 10$), it is faster than Quick Sort or Merge Sort because it has no recursion overhead.
- **Stable:** It preserves the relative order of equal elements.

3. Which sorting algorithm is easily adaptable to singly linked lists? Justify.

- **Algorithm: Merge Sort**.

- **Justification:**
    - Merge sort does not require random access (indexing), which Linked Lists do not support. It accesses data sequentially.
    - Merge operations in Linked Lists can be done by simply changing pointers, requiring $O(1)$ auxiliary space (unlike arrays which need $O(n)$ extra space for merging).
    - Quick Sort and Heap Sort rely heavily on random access (indices), making them inefficient for Linked Lists.

4. Tightest upper bound for worst-case performance of Quick Sort with central pivot.

- **Analysis:** Choosing the central element reduces the probability of hitting the worst case compared to choosing the first element (especially in sorted arrays).
- **Worst Case:** However, it is mathematically possible to construct an input where the central element is always the maximum or minimum of the current partition.
- **Answer:** The theoretical tightest upper bound remains **$O(n^2)$**, although it is much rarer in practice.

1. Bubble sort and selection sort have worst-case time complexity $O(n^2)$, but selection sort is faster than bubble sort. Why?

- **Number of Swaps:**
    - **Bubble Sort:** Can perform up to $O(n^2)$ swaps in the worst case (swapping neighbours continuously).
    - **Selection Sort:** Performs exactly $O(n)$ swaps (one swap per pass).
- **Memory Writes:** Writing to memory (swapping) is much more expensive than reading (comparing). Since Selection Sort minimizes swaps, it outperforms Bubble Sort.

2. Let P be a Quick Sort Program to sort numbers in ascending order using the rst element as the pivot. Let t1 and t2 be the number of comparisons made by P for the inputs 1, 2, 3, 4, 5 and 4, 1, 5, 3, 2 respectively. Which one of the following holds?

(A) t1 = 5 (B) t1 < t2 (C) t1 > t2 (D) t1 = t2

Option C (t1 > t2)

Note: For input 1 (sorted), the pivot is always the minimum, creating the worst-case scenario ($O(n^2)$ comparisons). For input 2 (random/mixed), the pivot 4 creates a balanced partition, reducing comparisons.

Tree

1. What is a binary tree?
(a) A tree with at most two children per node
(b) A tree with exactly two children per node
(c) A tree with at least two children per node
(d) A tree with no children

2. Which traversal method visits the root node first, then the left subtree, and finally the right subtree?
(a) In-order traversal
(b) Pre-order traversal
(c) Post-order traversal
(d) Level-order traversal

3. What is the maximum number of nodes in a binary tree of height h?
(a) $2^h - 1$
(b) 2^h
(c) $2^{h+1} - 1$
(d) h

4. Which of the following is true about a full binary tree?
(a) Every node has 0 or 2 children
(b) Every node has 2 children
(c) Every node has at least 2 children
(d) Every node has at most 1 child

5. Which tree traversal algorithm uses a queue data structure?
(a) Pre-order traversal
(b) Post-order traversal
(c) In-order traversal
(d) Level-order traversal

6. In a binary search tree, where are all the nodes with keys greater than the root key located?
(a) In the left subtree
(b) In the right subtree
(c) Anywhere in the tree
(d) At the root node

7. Which of the following trees is a height-balanced binary tree?
(a) AVL tree
(b) Binary search tree
(c) Full binary tree
(d) Complete binary tree

8. Which traversal method is used to print the nodes of a binary search tree in ascending order?
(a) Pre-order traversal
(b) Post-order traversal
(c) In-order traversal
(d) Level-order traversal

9. What is a complete binary tree?
(a) A tree in which all levels are fully filled
(b) A tree in which all levels except possibly the last are fully filled
(c) A tree with all nodes having at most one child
(d) A tree in which every node has 0 or 2 children

10. Which of the following properties does a red-black tree have?
(a) Every node is either red or black
(b) The root is always black
(c) Every leaf is black
(d) All of the above

11. Which type of binary tree has the property that every node has at most one child?
(a) Full binary tree
(b) Complete binary tree
(c) Skewed binary tree
(d) Perfect binary tree

12. Which tree traversal method is implemented using a stack data structure?
(a) Pre-order traversal
(b) Post-order traversal
(c) In-order traversal
(d) All of the above

13. What is the height of an empty tree?
(a) -1
(b) 0
(c) 1
(d) Infinity

14. In an AVL tree, what is the balance factor of a node?
(a) The height of the left subtree minus the height of the right subtree
(b) The height of the right subtree minus the height of the left subtree
(c) The number of nodes in the left subtree minus the number of nodes in the right subtree
(d) The number of nodes in the right subtree minus the number of nodes in the left subtree

15. What is the time complexity of searching for an element in a balanced binary search tree?
(a) O(1)
(b) O(log n)
(c) O(n)
(d) O(n log n)

16. Which of the following is not a self-balancing binary search tree?
(a) AVL tree
(b) Red-black tree
(c) B-tree
(d) Binary search tree

17. In a B-tree of order m, what is the maximum number of children a node can have?
(a) m – 1
(b) m
(c) m + 1
(d) 2m

18. Which of the following traversal methods is also known as depth-first traversal?
(a) In-order traversal
(b) Pre-order traversal
(c) Post-order traversal
(d) All of the above

19. In a max heap, which node contains the largest value?
(a) Root node
(b) Leftmost leaf node
(c) Rightmost leaf node
(d) Any internal node

20. What is the time complexity of inserting an element into a binary heap?
(a) O(1)
(b) O(log n)
(c) O(n)
(d) O(n log n)

1. Explain the concept of a tree. Discuss its applications.

- **Concept:**
  - A **Tree** is a non-linear data structure that represents a hierarchical relationship between elements.
  - It consists of a set of **nodes** connected by **edges**.
  - One specific node is designated as the **Root**.
  - The remaining nodes are partitioned into disjoint sets, each of which is a subtree of the root.
  - Common terms include Parent, Child, Leaf (node with no children), and Sibling.
- **Applications:**
  - **File Systems:** Operating systems use trees to organize files and directories.
  - **Hierarchical Data:** Storing organizational charts or HTML Document Object Models (DOM).
  - **Searching & Sorting:** Binary Search Trees (BST) and B-Trees allow for efficient searching (O(log n)) and sorting of data.
  - **Compilers:** Syntax Trees are used to parse arithmetic expressions and program syntax.
  - **Network Routing:** Algorithms like Spanning Tree Protocol (STP) prevent loops in network bridges.

2. What are the two ways of representing binary trees in memory? Which one do you prefer and why?

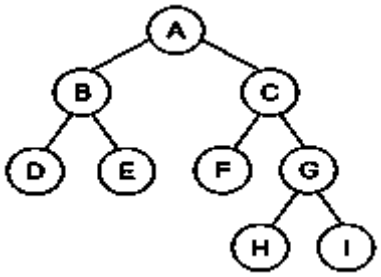
- **1. Array Representation (Sequential):**
  - Nodes are stored in a single contiguous array.
  - For a node at index i: Left child is at 2i + 1, Right child is at 2i + 2 (assuming 0-based indexing).
- **2. Linked Representation (Linked List):**
  - Uses dynamic memory allocation where each node contains three parts: a data field, a pointer to the left child, and a pointer to the right child.
- **Preference:**
  - **Linked Representation** is generally preferred.
  - **Reason:** It is more memory-efficient for arbitrary binary trees. Array representation wastes significant space (sparse array) if the tree is not a complete binary tree (e.g., a skewed tree). Linked lists also allow the tree to grow dynamically without the overhead of resizing an array.

3. Write short notes on:

- **Complete Binary Trees:** A binary tree in which every level, except possibly the last, is completely filled, and all nodes in the last level are as far left as possible.
- **Extended Binary Trees (2-Trees):** A binary tree where every node has either 0 or 2 children. The original nodes are called internal nodes, and special "dummy" nodes added to replace null pointers are called external nodes.

- **Expression Trees:** A specific type of binary tree used to represent arithmetic expressions. Leaves contain operands (constants/variables), and internal nodes contain operators (+, -, *, /). An in-order traversal yields the infix expression.

4. Consider the tree given below. Answer the following questions:



- **Name the leaf nodes:** D, E, F, H, I
- **Name the non-leaf nodes:** A, B, C, G
- **Name the ancestors of E:** B, A
- **Name the descendants of A:** B, C, D, E, F, G, H, I
- **Name the siblings of C:** B
- **Find the height of the tree:** 3 (Counting edges on the longest path: A -> C -> G -> H)
- **Find the height of subtree rooted at E:** 0 (E is a leaf)
- **Find the level of node E:** 2 (Assuming Root A is Level 0)
- **Traversals:**
  - **In-order (Left-Root-Right):** D, B, E, A, F, C, H, G, I
  - **Pre-order (Root-Left-Right):** A, B, D, E, C, F, G, H, I
  - **Post-order (Left-Right-Root):** D, E, B, F, H, I, G, C, A
  - **Level order:** A, B, C, D, E, F, G, H, I

5. Explain the concept of binary search trees (BST).

- A **Binary Search Tree** is a node-based binary tree data structure with the following properties:
  - The value of the **Left** child is strictly **less than** the parent node's value.
  - The value of the **Right** child is strictly **greater than** the parent node's value.
  - Both the left and right subtrees must also be binary search trees.
- This structure allows for binary search-like efficiency ($O(\log n)$) for lookup operations.

6. Explain the operations on binary search trees.

- **Search:** Start at the root. If the target is equal to the root, stop. If less, recurse left; if greater, recurse right.
- **Insertion:** Perform a search. When a NULL pointer is reached, insert the new node there.
- **Deletion:**

- *Case 1 (Leaf):* Simply remove the node.
- *Case 2 (One Child):* Remove the node and link its parent directly to its child.
- *Case 3 (Two Children):* Find the node's **In-order Successor** (smallest value in the right subtree), replace the node's value with the successor's value, and then delete the successor node.

7. Write a short note on threaded binary trees.

- A **Threaded Binary Tree** optimizes space by utilizing the null pointers usually present in leaf nodes.
- **"Threads"** are references used to point to the predecessor or successor nodes in a specific traversal order (usually In-order).
- **Right Null Pointers:** Point to the In-order Successor.
- **Left Null Pointers:** Point to the In-order Predecessor.

8. Why are threaded binary trees considered efficient? List the advantages.

- **Efficiency:** They allow for linear traversal of the tree without the need for a stack or recursion, which saves memory and reduces function call overhead.
- **Advantages:**
  - Faster traversal (approx. 50% faster in some implementations).
  - No auxiliary stack memory required for traversal.
  - Efficient determination of predecessor and successor nodes.

9. Discuss the advantages of an AVL tree.

- **Self-Balancing:** AVL trees strictly maintain a height difference (balance factor) of at most 1 between left and right subtrees.
- **Guaranteed Performance:** Because the tree remains balanced, the height is always $O(\log n)$. This guarantees that Search, Insert, and Delete operations always take logarithmic time, avoiding the $O(n)$ worst-case scenario of skewed BSTs.
- **Predictability:** Ideal for applications requiring consistent lookup times.

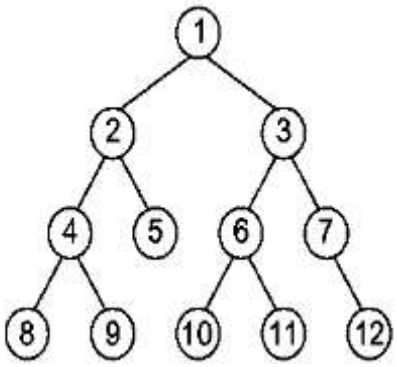
10. List the merits and demerits of a splay tree.

- **Merits:**
  - **Self-Optimizing:** Frequently accessed nodes move closer to the root, making future accesses to them very fast ($O(1)$). This is ideal for "locality of reference" scenarios (e.g., caches).
  - **Simpler Implementation:** Unlike AVL or Red-Black trees, Splay trees do not need to store extra bookkeeping data like balance factors or colours.
  - **Amortized Efficiency:** While a single operation can be slow, the *average* time over a sequence of m operations is guaranteed to be $O(m \log n)$.
- **Demerits:**
  - **Worst-Case Performance:** A single operation can take linear time $O(n)$ (e.g., accessing the deepest node in a skewed tree).

- **Not Thread-Safe:** Search operations (read-only in other BSTs) modify the tree structure (splaying), which complicates concurrent access.
- **Variable Height:** The tree can become completely linear (unbalanced) temporarily, unlike AVL trees which are strictly balanced.

11. Provide the memory representation of the binary tree given below and answer:

- **Find the result of one-way in-order, one-way pre-order, and two-way in-order threading.**
- **Draw the tree for each and give its memory representation.**



Base Traversals:

- **In-Order Sequence (Left, Root, Right):** 8, 4, 9, 2, 5, 1, 10, 6, 11, 3, 7, 12
- **Pre-Order Sequence (Root, Left, Right):** 1, 2, 4, 8, 9, 5, 3, 6, 10, 11, 7, 12

1. One-Way In-Order Threading (Right-Threaded)

In a right-threaded binary tree, a NULL right pointer is replaced by a pointer (thread) to the node's **In-Order Successor**.

- **Memory Structure:** Nodes contain LChild, Data, RChild, and an RTag.
  - If RTag == 0: RChild points to an actual child.
  - If RTag == 1: RChild is a **Thread** pointing to the successor.

Thread Connections (Right Pointer Changes): | Node | Original R-Ptr | Status | New Thread Destination (Successor) | | :--- | :--- | :--- | :--- | | **8** | NULL | Thread | **4** | | **9** | NULL | Thread | **2** | | **5** | NULL | Thread | **1** | | **10** | NULL | Thread | **6** | | **11** | NULL | Thread | **3** | | **7** | 12 (Link) | Link | *(Stays as Link to 12)* | | **12** | NULL | Thread | **NULL** (or Header) |

2. One-Way Pre-Order Threading

In a one-way pre-order threaded tree, a NULL right pointer is replaced by a pointer to the **Pre-Order Successor**.

Thread Connections (Right Pointer Changes): | Node | Original R-Ptr | Status | New Thread Destination (Successor) | | :--- | :--- | :--- | :--- | | **8** | NULL | Thread | **9** | | **9** | NULL | Thread | **5** | | **5** | NULL | Thread | **3** | | **10** | NULL | Thread | **11** | | **11** | NULL | Thread | **7** | | **12** | NULL | Thread | **NULL** (or Header) |

(Note: Standard links exist for nodes 1, 2, 4, 3, 6, and 7 as they define the tree structure.)

3. Two-Way In-Order Threading (Fully Threaded)

In a fully threaded tree, NULL Left pointers point to the **Predecessor**, and NULL Right pointers point to the **Successor**. This requires two boolean tags (LTag, RTag).

- **Memory Representation Schema:** [LTag | LChild | Data | RChild | RTag]

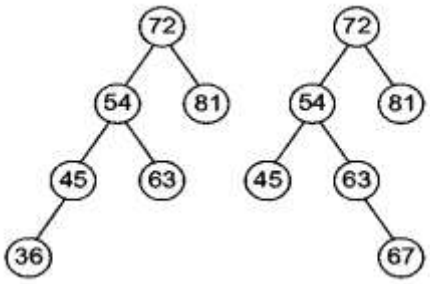
Thread Logic:

1. **Right Threads (Successors):** Same as Section 1 (8→4, 9→2, 5→1, 10→6, 11→3, 12→Head).
2. **Left Threads (Predecessors):**
    - **Node 8:** Left is NULL. Predecessor is **Head**. (Thread: 8 → Head)
    - **Node 9:** Left is NULL. Predecessor is **4**. (Thread: 9 → 4)
    - **Node 5:** Left is NULL. Predecessor is **2**. (Thread: 5 → 2)
    - **Node 10:** Left is NULL. Predecessor is **1**. (Thread: 10 → 1)
    - **Node 11:** Left is NULL. Predecessor is **6**. (Thread: 11 → 6)
    - **Node 7:** Left is NULL. Predecessor is **3**. (Thread: 7 → 3)
    - **Node 12:** Left is NULL. Predecessor is **7**. (Thread: 12 → 7)

***C*-Style Memory Representation (Struct):**

```
struct ThreadedNode {
    int data;
    struct ThreadedNode *left;
    struct ThreadedNode *right;
    int LTag; // 0 for Link, 1 for Thread
    int RTag; // 0 for Link, 1 for Thread
};
```

12. Balance the AVL trees given below:



- **Tree 1 (Left):**
    - **Structure:** Root 72 -> Left 54 -> Left 45. (Right subtrees exist but path 72-54-45 causes imbalance).
    - **Issue:** The node **72** has a Balance Factor of +2 (Left Height 2, Right Height 0). The left child (54) has a Balance Factor of +1 (Left heavy). This is an **LL Case**.
    - **Solution (Right Rotation at 72):**
        1. Node **54** becomes the new Root.

2. Node **45** remains the Left child of 54.
3. Node **72** becomes the Right child of 54.
4. Node **63** (originally right child of 54) becomes the Left child of 72.

- **Tree 2 (Right):**
  - **Structure:** Root 72 -> Left 54 -> Right 63.
  - **Issue:** The node **72** has a Balance Factor of +2. The left child (54) has a Balance Factor of -1 (Right heavy). This is an **LR Case**.
  - **Solution (Double Rotation):**
    1. **Left Rotation at 54:** Node 63 moves up, 54 moves down to left. (Sequence becomes 72-63-54).
    2. **Right Rotation at 72:** Node 63 moves up to Root, 72 moves down to right.
  - **Final Root:** 63. **Left Child:** 54. **Right Child:** 72.

13. Create an AVL tree using the following sequence of data: 16, 27, 9, 11, 36, 54, 81, 63, 72.

- **Step 1:** Insert 16.
- **Step 2:** Insert 27 (Right of 16).
- **Step 3:** Insert 9 (Left of 16). Tree: 9 <- 16 -> 27. Balanced.
- **Step 4:** Insert 11 (Right of 9). Imbalance at 9 (LR case). Rotate 9 Left -> 16 -> 11 -> 9 (incorrect). *Correct LR:* Left rotate 9 (9 becomes child of 11), Right rotate 16? No, imbalance is at subtree.
  - *Correction:* Tree is 16 (Root), 9 (L), 27 (R). Insert 11 (Right of 9). 16 is Bal (+2). LR Rotation on 16? No, 16 Bal is +1. 9 Bal is -1. Tree is balanced.
- **Step 5:** Insert 36 (Right of 27).
- **Step 6:** Insert 54 (Right of 36). **Imbalance at 27** (Right-Right Case).
  - Rotate Left on 27. Subtree becomes 36 (Root) -> 27 (L), 54 (R).
- **Step 7:** Insert 81 (Right of 54). **Imbalance at 16** (Root). Right height is 3, Left is 2.
  - Rotate Left on 16. New Root 36.
- **Step 8:** Insert 63.
- **Step 9:** Insert 72. **Imbalance at 81?** No. **Imbalance at 54**. Right-Left Case (54-81-63). Rotate 81 Right, then 54 Left.
- **Final Structure (Root 36):**
  - Left: 16 (Child: 9, 11, 27)
  - Right: 63 (Child: 54, 81(Child: 72))

14. Draw all possible binary search trees of 7, 9, and 11.

- The number of possible BSTs with N=3 unique keys is given by the Catalan Number:

$$\frac{1}{n+1}\binom{2n}{n} = 5.$$

- **Tree 1:** Root 7 -> R9 -> R11. (Right Skewed)

- **Tree 2:** Root 7 -> R11 -> L9.
- **Tree 3:** Root 9 -> L7, R11. (Balanced)
- **Tree 4:** Root 11 -> L7 -> R9.
- **Tree 5:** Root 11 -> L9 -> L7. (Left Skewed)

15. What is the maximum number of levels a binary search tree with 100 nodes can have?

- **Answer: 100 Levels.**
- **Reasoning:**
  - The "maximum" number of levels occurs when the Binary Search Tree (BST) is completely **skewed** (degenerate).
  - In a skewed tree, every node has exactly one child (except the leaf). This essentially behaves like a Linked List.
  - Therefore, if there are N nodes, the maximum number of levels is N.
  - For N = 100, Maximum Levels = 100.

16. What is the maximum height of a tree with 32 nodes?

- **Answer: 31** (assuming height is defined by edges).
- **Reasoning:**
  - The maximum height occurs in a **skewed binary tree** (Left-skewed or Right-skewed).
  - **Definition of Height:** In most standard Data Structure curriculums (like CLRS), height is defined as the number of **edges** on the longest path from the root to a leaf.
  - **Formula:** $Height_{max} = N - 1$
  - **Calculation:** 32 - 1 = 31.
  - *(Note: If your specific curriculum defines height as the number of nodes on the path, the answer is 32).*

17. What is the maximum number of nodes at levels 3, 4, and 12 in a binary tree?

- **Answer:**
  - **Level 3:** 4 nodes
  - **Level 4:** 8 nodes
  - **Level 12:** 2048 nodes
- **Reasoning:**
  - The maximum number of nodes at any given level L in a binary tree is calculated using the formula: 2^{L-1} (assuming the Root is at Level 1).
  - **For Level 3:** $2^{3-1} = 2^2 = 4$.
  - **For Level 4:** $2^{4-1} = 2^3 = 8$.
  - **For Level 12:** $2^{12-1} = 2^{11} = 2048$.

18. Draw all possible non-similar binary trees with three nodes.

- **Answer:** There are **5** possible structurally distinct (non-similar) binary trees with 3 nodes. This is determined by the **Catalan Number** formula: $C_n = \frac{1}{n+1}\binom{2n}{n}$. For n=3, the result is 5.

- **Visual Representation:**

1. Right Skewed:

(Root)
  \
  (Child)
    \
    (Child)

2. Left Skewed:

    (Root)
    /
  (Child)
  /
(Child)

3. Root with Balanced Children:

    (Root)
    / \
(Child) (Child)

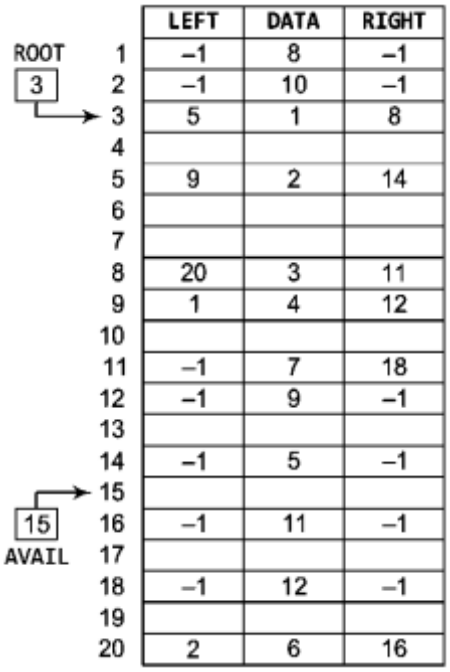
4. Left-Right Zigzag:

  (Root)
  /
(Child)
  \
  (Child)

5. Right-Left Zigzag:

(Root)
  \
  (Child)
  /
(Child)

19. Draw the binary tree for the following memory representation:



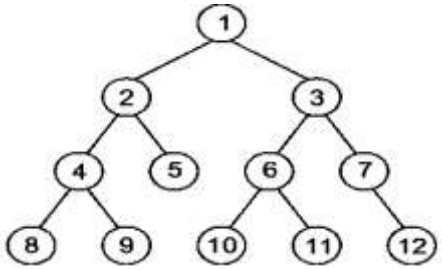
	LEFT	DATA	RIGHT
1	–1	8	–1
2	–1	10	–1
3	5	1	8
4			
5	9	2	14
6			
7			
8	20	3	11
9	1	4	12
10			
11	–1	7	18
12	–1	9	–1
13			
14	–1	5	–1
15			
16	–1	11	–1
17			
18	–1	12	–1
19			
20	2	6	16

1. **Level 0 (Root):**
    - Start at Index 3. **Data: 1**.
    - Left Ptr: 5. Right Ptr: 8.
2. **Level 1:**
    - **Left Child (Index 5): Data: 2**.
        - Left Ptr: 9. Right Ptr: 14.
    - **Right Child (Index 8): Data: 3**.
        - Left Ptr: 20. Right Ptr: 11.
3. **Level 2:**
    - **From Node 2 (Index 9): Data: 4**.
        - Left Ptr: 1. Right Ptr: 12.
    - **From Node 2 (Index 14): Data: 5**.
        - Left Ptr: -1. Right Ptr: -1. (Leaf Node)
    - **From Node 3 (Index 20): Data: 6**.
        - Left Ptr: 2. Right Ptr: 16.
    - **From Node 3 (Index 11): Data: 7**.
        - Left Ptr: -1. Right Ptr: 18.
4. **Level 3 (Leaves):**
    - **From Node 4 (Index 1): Data: 8**. (Leaf)
    - **From Node 4 (Index 12): Data: 9**. (Leaf)
    - **From Node 6 (Index 2): Data: 10**. (Leaf)
    - **From Node 6 (Index 16): Data: 11**. (Leaf)
    - **From Node 7 (Index 18): Data: 12**. (Leaf)

Visual Tree Representation

```
          1
        /   \
       2     3
      / \   / \
     4  5  6  7
    / \  / \  \
   8  9 10 11  12
```

20. Draw the memory representation of the binary tree given below



For this solution, we assume:

- NULL indicates no child exists (represented as 0 or NULL).
- Arbitrary memory addresses are assigned (e.g., Node 1 is at address 100) to demonstrate the linking logic.

Memory Address	Left Pointer (Address)	Data (Value)	Right Pointer (Address)
100	200	**1**	300
200	400	**2**	500
300	600	**3**	700
400	800	**4**	900
500	NULL	**5**	NULL
600	1000	**6**	1100
700	NULL	**7**	1200
800	NULL	**8**	NULL
900	NULL	**9**	NULL
1000	NULL	**10**	NULL
1100	NULL	**11**	NULL
1200	NULL	**12**	NULL

21. What is the difference between a graph and a tree?

- **Definition & Structure:**
  - **Tree:** A tree is a hierarchical data structure. It is a specialized form of a graph that is connected and **acyclic** (contains no loops). It has a specific "Root" node.

    - **Graph:** A graph is a network model consisting of vertices (nodes) and edges. It can be cyclic (contain loops) or acyclic, and connected or disconnected. It does not strictly require a root node.
- **Key Differences Table:**

Feature	Tree	Graph
Path	There is exactly one unique path between any two nodes.	There can be multiple paths (or no path) between nodes.
Edges	If a tree has N nodes, it has exactly N-1 edges.	The number of edges is not fixed relative to nodes.
Loops	**No loops/cycles** allowed.	Can contain loops/cycles.
Traversal	Techniques: Pre-order, In-order, Post-order, Level-order.	Techniques: BFS (Breadth-First Search), DFS (Depth-First Search).
Model	Hierarchical model (Parent-Child relationship).	Network model (Neighbour relationship).

22. What is a complete binary tree, and what is a full binary tree?

- **Full Binary Tree (Strict Binary Tree):**
    - A binary tree is **Full** if every node has either **0** or **2** children.
    - No node exists with exactly one child.
    - *Example:* Used in Huffman Coding trees.
- **Complete Binary Tree:**
    - A binary tree is **Complete** if all levels are completely filled except possibly the last level.
    - In the last level, all nodes must be as far **left** as possible.
    - *Key Property:* This structure allows the tree to be efficiently represented using an **Array** without gaps, which is the basis for the **Heap** data structure.

24. List some applications of trees.

- **File Systems:** Operating systems (like Linux or Windows) use tree structures to manage directories and files (Folders inside Folders).
- **Hierarchical Data Organization:** Used in organizational charts, taxonomies, and DOM (Document Object Model) for HTML web pages.
- **Compilers:** Syntax Trees (Abstract Syntax Trees) are used to parse code and validate the syntax of programs.
- **Searching and Indexing:**
    - **Binary Search Trees (BST):** For efficient searching ($O(\log n)$).
    - **B-Trees & B+ Trees:** Widely used in **Database** indexing to manage vast amounts of data on disks.
- **Network Routing:** Trees (Prefix Trees) are used in routers to store IP routing tables efficiently.
- **Compression:** Huffman Coding Trees are used in data compression algorithms (like ZIP files).

25. The height of a binary tree is the maximum number of edges in any root-to-leaf path. What is the maximum number of nodes in a binary tree of height h?

- **Answer:** $2^{h+1} - 1$
- **Detailed Explanation:**
  - **Level Capacity:** In a binary tree, the maximum number of nodes at any level i is 2^i (where the root is at level 0).
    - Level 0: $2^0 = 1$ node
    - Level 1: $2^1 = 2$ nodes
    - Level 2: $2^2 = 4$ nodes
    - ...
    - Level h: 2^h nodes
  - Total Sum: To find the maximum total nodes (N), we sum the capacity of all levels from 0 to h.

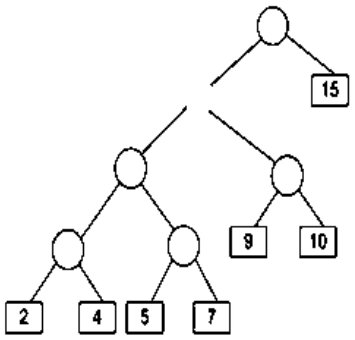
$$N = \sum_{i=0}^{h} 2^i = 2^0 + 2^1 + 2^2 + \cdots + 2^h$$

  - Geometric Series Formula: This is a geometric series with ratio r=2. The sum is:

$$N = \frac{1(2^{h+1} - 1)}{2 - 1} = 2^{h+1} - 1$$

  - **Example:** For a tree of height 2 (Root -> Children -> Leaves), max nodes = $2^{2+1} - 1 = 8 - 1 = 7$.

26. What is the weighted external path length of the binary tree shown below?



- **Answer: 144**
- **Definition:**

The Weighted External Path Length (WEPL) is the sum of the weighted depths of all external nodes (leaves).

$$WEPL = \sum (Value\ of\ Leaf_i \times Depth\ of\ Leaf_i)$$

(Note: Depth is defined as the number of edges from the root).

- Step-by-Step Calculation:

Let's trace the depth of each leaf node (square nodes) from the diagram:

1. **Leaf 15:** Direct right child of the root.
   - Depth: **1**
2. **Leaves 9 and 10:** Located in the right sub-branch of the left subtree.
   - Path: Root -> Left -> Right -> Leaf.

        - Depth: **3**
    3. **Leaves 2, 4, 5, and 7:** Located in the deeper left sub-branch.
        - Path: Root -> Left -> Left -> Parent -> Leaf.
        - Depth: **4**
- **Calculation Table:**

Leaf Value (W)	Depth (D)	Weighted Path (W×D)
15	1	15 x 1 = 15
9	3	9 x 3 = 27
10	3	10 x 3 = 30
2	4	2 x 4 = 8
4	4	4 x 4 = 16
5	4	5 x 4 = 20
7	4	7 x 4 = 28
TOTAL		**144**

- **Result:** The weighted external path length is **144**.

27. Detailed Definitions:

Part A: General Tree Concepts

1. **Non-Linear Data Structure:**
    - A data structure where elements are not arranged sequentially (like arrays or linked lists) but are organized in a hierarchical or interconnected manner.
    - **Key Attribute:** A single level can be connected to multiple elements at the next level (1:N relationship). Examples include Trees and Graphs.
2. **Tree:**
    - A tree is a hierarchical data structure consisting of a collection of nodes connected by directed edges.
    - **Key Attribute:** It has exactly one root node, and there are no cycles (loops) in the structure. For N nodes, there are N-1 edges.
3. **Node:**
    - A node is the fundamental building block of a tree or graph that stores data (value/key).
    - **Key Attribute:** In C, it typically contains the data element and pointers (references) to other nodes (children or neighbours).
4. **Root:**
    - The topmost node in a tree hierarchy.
    - **Key Attribute:** It is the only node in the tree that has no parent. All other nodes can be reached from the root via a unique path.
5. **Forest:**
    - A forest is a collection of one or more disjoint (non-connected) trees.
    - **Key Attribute:** If you remove the root node of a tree, the remaining unconnected sub-trees form a forest.
6. **Orchard:**
    - An orchard is strictly defined as a forest of **ordered** trees.

- **Key Attribute:** In an orchard, the relative order of the trees matters (e.g., First tree, Second tree), unlike a standard forest where order is irrelevant.

Part B: Node Relationships & Classifications

7. **Siblings:**
   - Nodes that share the exact same parent node.
   - **Example:** In a binary tree, the left child and right child of a specific node are siblings.
8. **Ancestors:**
   - All nodes that appear along the path from the Root to a specific node X.
   - **Key Attribute:** This includes the parent, grandparent, great-grandparent, up to the root.
9. **Descendant:**
   - Any node that is in the subtree of a specific node X.
   - **Key Attribute:** This includes children, grandchildren, and all subsequent nodes down to the leaves.
10. **Terminal Nodes (Leaf Nodes):**
    - Nodes that do not have any children (out-degree is 0).
    - **Key Attribute:** In memory representation, their child pointers are NULL.
11. **Non-Terminal Nodes (Internal Nodes):**
    - Nodes that have at least one child (degree ≥ 1).
    - **Key Attribute:** These are the intermediate nodes connecting the root to the leaves.

Part C: Measurements & Metrics

12. **Degree of a Node:**
    - The total number of subtrees (or children) attached to a specific node.
    - **Example:** In a binary tree, a node can have a degree of 0, 1, or 2.
13. **Degree of a Tree:**
    - The maximum degree of any node found within the tree.
    - **Example:** If a tree has a node with 3 children, and no node has 4, the degree of the tree is 3 (Ternary Tree).
14. **Level:**
    - The rank or generation of a node within the hierarchy.
    - **Key Attribute:** The Root is typically at Level 0 (or 1 depending on convention). Children of the root are at Level 1, and so on.
15. **Height:**
    - The number of edges on the longest path from the root to a leaf node.
    - **Key Attribute:** A tree with a single node (Root) has a height of 0.
16. **Path:**
    - A specific sequence of consecutive nodes and edges connecting a starting node to a destination node.
    - **Key Attribute:** In a tree, there is exactly one unique path between any two nodes.
17. **Internal Path:**

- The sum of the path lengths (depths) of all **internal (non-leaf)** nodes in the tree.
- **Usage:** Used to analyse the average search time for successful searches in a BST.

18. **External Path:**
    - The sum of the path lengths (depths) of all **external (leaf)** nodes in the tree.
    - **Usage:** Used to analyse the cost of unsuccessful searches in a BST.

Part D: Graph Theory Concepts

19. **Directed Graph (Digraph):**
    - A graph where the edges have a specific direction (represented by arrows).
    - **Key Attribute:** An edge (A, B) allows travel from A to B, but not necessarily B to A.
20. **Undirected Graph:**
    - A graph where edges are bidirectional and have no specific orientation.
    - **Key Attribute:** An edge (A, B) implies a connection from A to B and B to A equally.
21. **Weighted Graph:**
    - A graph where each edge is assigned a numerical value (weight).
    - **Usage:** Used to represent distances, costs, or bandwidths in algorithms like Dijkstra's Shortest Path.
22. **Sub-graph:**
    - A graph G' whose vertices and edges are subsets of another graph G.
    - **Key Attribute:** Every edge in the subgraph must also exist in the original graph connecting the same vertices.
23. **Degree of a Graph:**
    - In an undirected graph, it is the sum of the degrees of all its vertices.
    - **Handshaking Lemma:** The sum of degrees is exactly twice the number of edges ($2 \times |E|$).
24. **Cut Vertex (Articulation Point):**
    - A vertex whose removal increases the number of connected components in the graph.
    - **Key Attribute:** Removing this vertex disconnects the graph or breaks it into pieces.
25. **Pendant Vertex:**
    - A vertex in a graph that has a degree of exactly 1.
    - **Key Attribute:** It is connected to the rest of the graph by a single edge (often called a leaf in graph context).
26. **Clique:**
    - A subset of vertices in an undirected graph such that every two distinct vertices in the clique are adjacent.
    - **Key Attribute:** It represents a "Complete Subgraph."
27. **Complete Graph:**

- A simple graph where every pair of distinct vertices is connected by a unique edge.
- **Formula:** A complete graph with n vertices has n(n-1)/2 edges.

28. **Connected Graph:**
    - A graph where there is a path between every pair of vertices.
    - **Key Attribute:** No node is isolated or unreachable.
29. **Graph Isomorphism:**
    - Two graphs which contain the same number of graph vertices connected in the same way are said to be isomorphic.
    - **Key Attribute:** They may look different when drawn, but they have identical structural properties.

28. Short Notes

A. Binary Search Tree (BST)

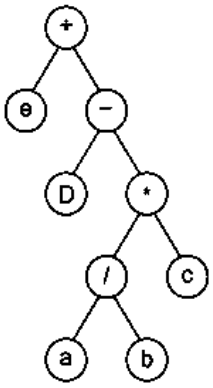
- **Definition:** A BST is a binary tree where for every node, the values in the left subtree are smaller, and values in the right subtree are larger.
- **Time Complexity:** Searching, Insertion, and Deletion take O(h) time, where h is the height.
- **Key Application:** Used for efficient searching and sorting (In-order traversal of a BST gives a sorted list).

B. Threaded Binary Tree

- **Definition:** A variation of a binary tree that replaces NULL pointers with "threads" (pointers) to the node's in-order predecessor or successor.
- **Key Benefit:** It allows for traversal of the tree (like In-order) without using a stack or recursion, saving memory overhead.
- **Types:** Single Threaded (Right pointer used) and Double Threaded (Left and Right pointers used)

29. For the expression tree given below, answer:

- **Extract the infix expression**
- **Find the prefix and postfix expressions**
- **Evaluate the infix expression with values a = 30, b = 10, c = 2, d = 30, e = 10**



A. Extract the Infix Expression

The **In-order Traversal** (Left -> Root -> Right) produces the standard mathematical notation (Infix). We add parentheses to preserve the hierarchy of operations defined by the tree structure.

- **Traversal:** e -> + -> (D -> - -> ((a -> / -> b) -> * -> c))
- Result:

e + (D - ((a / b) * c))

B. Find the Prefix and Postfix Expressions

1. Prefix Expression (Pre-order Traversal)

(Root -> Left -> Right)

- Start at Root: +
- Visit Left: e
- Visit Right Subtree (Root -): -
  - Visit Left (D): D
  - Visit Right (*): *
    - Visit Left (/): /
      - Visit Left (a): a
      - Visit Right (b): b
    - Visit Right (c): c
- Result:

+ \ e \ - \ D \ * \ / \ a \ b \ c

2. Postfix Expression (Post-order Traversal)

(Left -> Right -> Root)

- Visit Left: e
- Visit Right Subtree:
  - Visit Left (D): D
  - Visit Right (*):
    - Visit Left (/): a b /
    - Visit Right (c): c
    - Visit Root (*): *
  - Visit Root (-): -
- Visit Root (+): +
- **Result:**

e \ D \ a \ b \ / \ c \ * \ - \ +

C. Evaluate the Infix Expression

Given Values:

- a = 30
- b = 10
- c = 2
- D = 30 (Assuming 'D' in tree corresponds to 'd' in the question text)
- e = 10

Step-by-Step Calculation:

1. Substitute values into the Infix Expression:

10 + (30 - ((30 / 10) * 2))

2. Evaluate the innermost parenthesis (Division):

30 / 10 = 3

New Expression: 10 + (30 - (3 * 2))

3. Evaluate the next operation (Multiplication):

3 * 2 = 6

New Expression: 10 + (30 - 6)

4. Evaluate the subtraction:

30 - 6 = 24

New Expression: 10 + 24

5. Final Addition:

10 + 24 = 34

30. Convert the pre x expression /ab*+bcd into in x, and draw the corresponding expression tree.

Step	**Symbol Scanned (R → L)**	**Action**	**Stack Content (Bottom → Top)**
1	d	Push operand	[d]
2	c	Push operand	[d, c]
3	b	Push operand	[d, c, b]
4	+	Pop b (left), c (right). Combine. Push (b+c).	[d, (b+c)]
5	*	Pop (b+c) (left), d (right). Combine. Push ((b+c)*d).	[((b+c)*d)]
6	b	Push operand	[((b+c)*d), b]
7	a	Push operand	[((b+c)*d), b, a]
8	/	Pop a (left), b (right). Combine. Push (a/b).	[((b+c)*d), (a/b)]

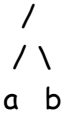
The resulting infix expressions are:

1. **a / b**
2. **(b + c) * d**

2. Expression Trees

Since the prefix expression resolves into two separate expressions, we draw an expression tree for each.

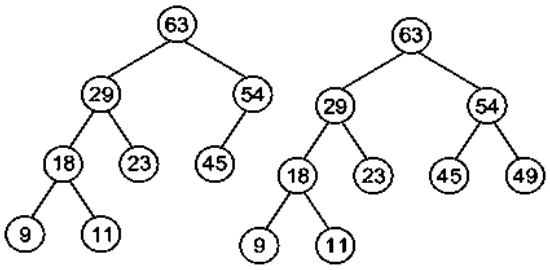
Tree 1: For the expression a / b (Prefix: /ab)



Tree 2: For the expression (b + c) * d (Prefix: *+bcd)

```
      *
     / \
    +   d
   / \
  b   c
```

31. Determine if the trees below are complete or full binary trees



Analysis of Tree 1 (Left Tree)

- **Structure:**
  - **Root:** 63
  - **Internal Nodes:** 29 (2 children), 54 (1 child: 45), 18 (2 children).
  - **Leaves:** 9, 11, 23, 45.

1. **Is it a Full Binary Tree?**
   - **Definition:** A Full Binary Tree requires every node to have either **0** or **2** children.
   - **Verdict: NO**.
   - **Reason:** Node **54** has exactly **one child** (node 45). This violates the property of a full binary tree.
2. **Is it a Complete Binary Tree?**
   - **Definition:** A Complete Binary Tree must have all levels completely filled, except possibly the last, which must be filled from left to right without gaps.
   - **Verdict: NO**.
   - **Reason:** Level 2 is not fully filled (Node 54 is missing its right child), yet there are nodes at Level 3 (9 and 11). In a complete tree, you cannot start a new level (Level 3) until the previous level (Level 2) is entirely full.

Analysis of Tree 2 (Right Tree)

- **Structure:**
  - **Root:** 63
  - **Internal Nodes:** 29 (2 children), 54 (2 children), 18 (2 children).
  - **Leaves:** 9, 11, 23, 45, 49.

1. **Is it a Full Binary Tree?**
   - **Verdict: YES**.

- **Reason:** Every node in this tree has either **0 children** (leaves: 9, 11, 23, 45, 49) or **2 children** (internal nodes: 18, 29, 54, 63). No node has only 1 child.

2. **Is it a Complete Binary Tree?**
   - **Verdict: YES**.
   - **Reason:**
     - **Level 0, 1, and 2** are completely filled with the maximum number of nodes (2^0, 2^1, 2^2).
     - **Level 3** (the last level) contains nodes 9 and 11. These nodes are positioned at the **leftmost** spots available.
     - There are no gaps between the nodes at the last level, and no deeper levels exist.

32. List all possible non-similar binary trees with four nodes.

- **Answer:** There are **14** possible non-similar binary trees.
- Calculation:

The number of structurally unique binary trees with n nodes is given by the Catalan Number formula:

C_n = \frac{1}{n+1}\binom{2n}{n}

For $n = 4$:

$$C_4 = \frac{1}{4+1}\binom{8}{4} = \frac{1}{5} \times \frac{8 \times 7 \times 6 \times 5}{4 \times 3 \times 2 \times 1} = \frac{1}{5} \times 70 = 14$$

---

5. Construct the expression tree for the expression: $E = (2a + 5b)(x - 7y)^4$.

(Note: The provided text "(x 7y)4" was interpreted as $(x - 7y)^4$ or a similar multiplication interaction to create a valid expression tree structure).

- **Infix Expression:** $((2 * a) + (5 * b)) * ((x - (7 * y)) \uparrow 4)$
- **Tree Structure:**
  - The root is the multiplication * connecting the two main terms.
  - **Left Subtree:** Represents 2a + 5b. Root is +.
    - Left child is * (2, a).
    - Right child is * (5, b).
  - **Right Subtree:** Represents (x - 7y)^4. Root is ^ (Power).
    - Left child is - (x, *).
      - Right child of - is * (7, y).
    - Right child is 4.

---

6. What is an AVL tree? How do AVL trees differ from binary search trees? Explain the rotations and balance factors.

AVL Tree Definition:

An AVL (Adelson-Velsky and Landis) tree is a self-balancing Binary Search Tree (BST). It ensures that the tree remains approximately balanced to guarantee logarithmic time complexity for operations.

Difference from Standard BST:

- **Height Control:** A standard BST can become skewed (like a linked list) in the worst case (e.g., inserting sorted data), leading to $O(n)$ complexity. An AVL tree strictly maintains height balance, guaranteeing $O(\log n)$ for search, insertion, and deletion.
- **Balance Property:** In an AVL tree, for every node, the height difference between its left and right subtrees cannot exceed 1.

Balance Factor (BF):

- The balance factor of a node is calculated as:

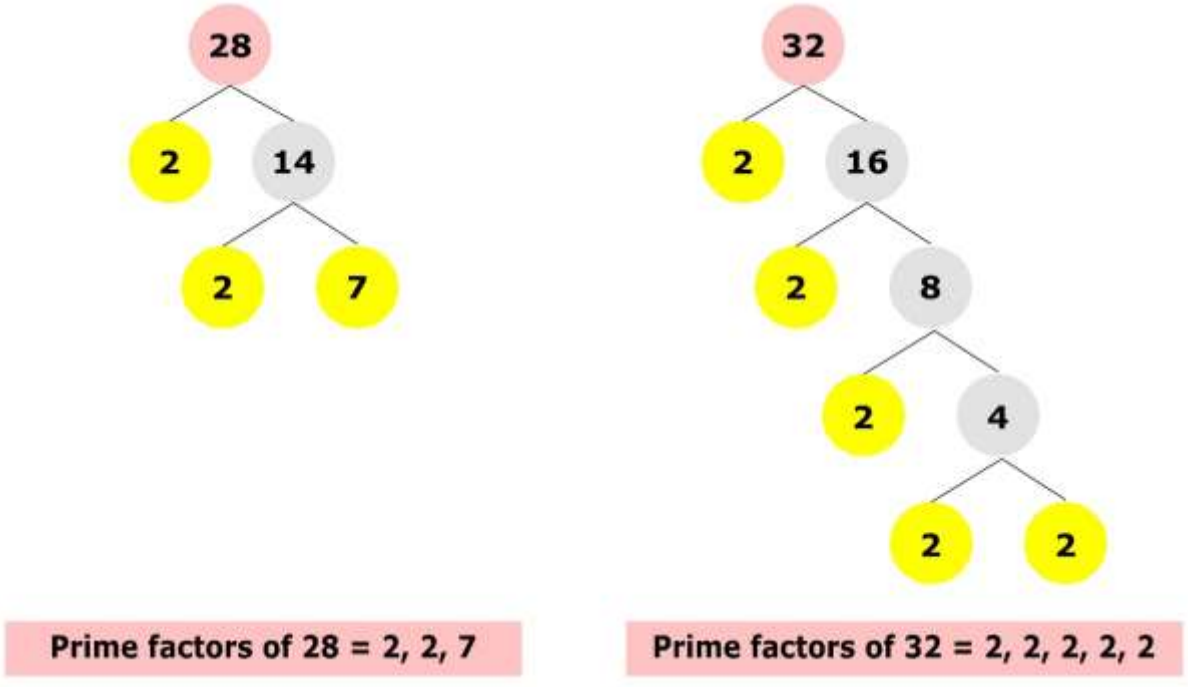
$$BF = Height(Left Subtree) - Height(Right Subtree)$$

- Allowed values in AVL: **{-1, 0, 1}**. If $|BF| > 1$, re-balancing is required.

Rotations:

Rotations are operations used to restore balance.

1. **LL Rotation (Single):** Performed when a node is inserted into the left subtree of the left child of a critically unbalanced node.
2. **RR Rotation (Single):** Performed when a node is inserted into the right subtree of the right child.
3. **LR Rotation (Double):** Left rotation on the child, followed by Right rotation on the parent. Used when insertion is in the right subtree of a left child.
4. **RL Rotation (Double):** Right rotation on the child, followed by Left rotation on the parent. Used when insertion is in the left subtree of a right child.



Shutterstock

Explore

---

7. Prove $E = I + 2n$ for a binary tree with external path length E and internal path length I.

*(Clarification: In standard theorems, n usually refers to the number of **internal nodes**. The formula connects Internal Path Length I, External Path Length E, and internal nodes n.)*

Proof by Induction:
Let n be the number of internal nodes.

1. **Base Case ($n=0$):**
    - A tree with 0 internal nodes is a single leaf (root).
    - $I = 0$ (depth of root is 0).
    - $E = 0$ (depth of single leaf is 0).
    - Formula: $0 = 0 + 2(0)$. Holds true.
2. **Inductive Hypothesis:**
    - Assume $E = I + 2n$ holds for a tree with n internal nodes.
3. **Inductive Step:**
    - Consider a tree with n internal nodes. Select a leaf at depth k and replace it with a new internal node having two leaf children.
    - **New Internal Nodes:** $n' = n + 1$.
    - **Change in I:** The chosen leaf becomes internal at depth k. So, $I' = I + k$.
    - Change in E: We lost a leaf at depth k, but gained two leaves at depth $k+1$.

$$E' = E - k + 2(k+1) = E - k + 2k + 2 = E + k + 2$$

    - Substitute $E = I + 2n$:

$$E' = (I + 2n) + k + 2$$
$$E' = (I + k) + 2n + 2$$
$$E' = I' + 2(n + 1)$$
$$E' = I' + 2n'$$

    - **Conclusion:** The relation holds for $n+1$. Q.E.D.

---

8. For a non-empty binary tree, prove $n_0 = n_2 + 1$.

- **Definitions:**
    - n_0: Number of leaf nodes (degree 0).
    - n_2: Number of nodes with degree 2.
    - n_1: Number of nodes with degree 1.
    - N: Total nodes $= n_0 + n_1 + n_2$.

Proof using Edges:

1. Edge Count by Nodes: In any tree with N nodes, there are $N-1$ edges (every node has an incoming edge except the root).

$$Edges = n_0 + n_1 + n_2 - 1$$

2. **Edge Count by Degrees:** The number of edges is also the sum of outgoing branches.
    - Leaves (n_0) contribute 0 edges.
    - Degree 1 nodes (n_1) contribute $1 \times n_1$ edges.
    - Degree 2 nodes (n_2) contribute $2 \times n_2$ edges.

$$Edges = n_1 + 2n_2$$

3. Equating the two:

$$n_0 + n_1 + n_2 - 1 = n_1 + 2n_2$$

4. Simplify:

Subtract n_1 from both sides:

$$n_0 + n_2 - 1 = 2n_2$$
Subtract n_2 from both sides:
$$n_0 - 1 = n_2 \implies n_0 = n_2 + 1$$
Q.E.D.

Questions for Bloom's Taxonomy Level: Apply

1. Insert 7, 5, 1, 8, 3, 6, 0, 9, 4, 2 into an empty BST. What is the in-order traversal?

- **Answer: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9**
- **Reasoning:** The In-Order traversal of **any** Binary Search Tree always yields the values in strictly increasing sorted order, regardless of the insertion sequence.

2. What is the maximum possible number of nodes in a binary tree at level 6?

- **Answer: 32** (Assuming Root is at Level 1).
- **Formula:** Max nodes at level $L = 2^{L-1}$.
- Calculation: $2^{6-1} = 2^5 = 32$.

(Note: If the convention defines Root at Level 0, the answer is $2^6 = 64$. Level 1 is the standard academic convention).

3. Pre-order: 30, 20, 10, 15, 25, 23, 39, 35, 42. Find Post-order.

- **Reconstructed Tree:**
  - Root: 30
  - Left Child: 20 $\rightarrow$ Children: 10 (Left), 25 (Right).
    - 10 has Right Child: 15.
    - 25 has Left Child: 23.
  - Right Child: 39 $\rightarrow$ Children: 35 (Left), 42 (Right).
- Post-Order Traversal (Left, Right, Root):

15, 10, 23, 25, 20, 35, 42, 39, 30

4. BST Operations (Construction and Deletion):

- **Final Deletion Analysis:**
  - **Delete 23:** 23 is a leaf (or has 1 child depending on duplicate handling). Simply remove.
  - **Delete 56:** Internal node. Replaced by In-order successor or predecessor.
  - **Delete 2:** Leaf node. Simply remove.
  - **Delete 45:** Leaf node (after insertion of 45). Simply remove.
- Resulting Tree Content (In-Order):

Deleting the specified values removes them from the sorted sequence.
Remaining values: 4, 12, 21, 32, 39, 46, 48, 54, 55, 63, 67, 87, 98.